

Abstract

The protection of a network is one of the main challenges for IT security professionals in a interconnected computer environment. The detection of an attack and the corresponding response times can be crucial in terms of information leakage and availability. For this reason Network Intrusion Detection Systems (NIDSs) are often used to detect or classify any malicious behaviour. Those systems are often based on static rules, defined by professionals in the security area. However the attackers and their action rise in quantity, quality and variety making it harder for a team to gasp all the possibilities and define them in a rule based policy. The need for a more adaptive solution rises.

The interest in machine learning algorithms recently is founded by good results in areas where it is hard to classify input based on static rules and algorithms. The emergent nature of Artificial Neural Networks (ANNs) enables the learning without such restrictions only based on given input data and labels. This can also be used in a NIDS to define a learning systems to classify behaviours in a computer network. But the complexity shifts since those systems are like other ANN based classifiers vulnerable to adversarial examples, which propose a security thread since they have the potential to impact the NIDSs performance in a negative way. For this reason the need of protection against those vectors arises and with it the need of a framework to test such behaviours. In this thesis Network Anomaly Detection for Industrial Control Systems (NADICS) was extended with new functionalities to train Deep Neural Networks (DNNs), attack them with multiple adversarial attacks and perform measurements to harden the networks against such. For this purpose multiple datasets from conceptional different sources with different classification scopes will be used.

Kurzfassung

Der Schutz eines Computernetzwerks ist eins der Hauptherausforderungen für IT Sicherheitspersonal. Die Detektierung einer Attacke und die entsprechenden Reaktionszeiten können eine kritische Rolle dabei spielen ob Information abfließen oder ein System versagt. Aus diesem Grund kommen oft NIDSs Systeme zum Einsatz. Diese sollen schädliches Verhalten detektieren und klassifizieren. Da diese Systeme oft auf statischen Regeln basieren, sind sie auf die Expertise von Menschen angewiesen. Das wird zu einer Herausforderung da die Attacken sich in Quantität und Qualität weiter entwickeln. Das macht es für ein IT Sicherheitsteam schwer alle Facetten zu erfassen. Aus diesem Grund werden adaptive Regeln benötigt.

Das kürzlich aufkommende Interesse in Machine Learning (ML) ist mit immer besser werdenden Leistungen verbunden. Die emergente Natur von ANNs erlaubt es dem Computer selbstständig Aufgaben zu erlernen die schwer in einem Programmcode beschrieben werden können. Diese Funktionalität kann auch benutzt werden um Netzwerkverkehr zu klassifizieren. Doch die Komplexität bei diesem Szenario wird auf die ANNs übertragen. Denn diese sind gegen sogenannte Adversarial Attacks anfällig. Diese können gezielt dazu benutzt werden um eine Misklassifizierung hervor zu rufen und somit einen Angriff zu verschleiern. Aus diesem Grund ist es notwendig diese Formen der Attacken zu erforschen.

In dieser Arbeit werden mehrere Datensätze ermittelt und mit diesen ANNs trainiert um diese im Rahmen von Adversarial Attacks zu untersuchen.

Contents

Abstract	iii
Kurzfassung	iv
1. Introduction	1
1.1. Overview	1
1.2. Why use an IDS	1
1.3. Why utilize Machine Learning	2
1.4. Why protection against adversarial attacks	2
1.5. Research questions	2
1.6. Structure	3
2. Background	4
2.1. Networking	4
2.1.1. Concept	4
2.1.2. Transport	5
2.1.3. Internet protocol	5
2.1.4. Firewalls	6
2.1.5. Intrusion detection systems	6
2.1.6. Signature detectors	8
2.1.7. Compound detectors	8
2.1.8. Counter measurements	8
2.2. Machine Learning	9
2.2.1. Supervision	9
2.2.2. Reinforcement Learning (RL)	9
2.2.3. Regression and classification	9
2.2.4. Overfitting	10
2.2.5. Performance metrics	11
2.2.6. K fold cross validation	13
2.2.7. No free lunch theorem	13
2.2.8. Feature Selection	13
2.2.9. Principal Component Analysis	14
2.3. Classical ML Algorithms	14
2.3.1. Gaussian Naive Bayes	14
2.3.2. K-means Clustering	15
2.3.3. Tree based models	16

2.3.4.	AdaBoost	17
2.3.5.	Gradient boosting	17
2.3.6.	k-Nearest Neighbors	18
2.3.7.	Support Vector Machines (SVM)	20
2.4.	Artificial Neural Networks	20
2.4.1.	Dimensionality	22
2.4.2.	Data Availability	23
2.5.	Adversarial Machine Learning Attack	24
2.5.1.	fast gradient sign method (FGSM)	26
2.5.2.	Basic Iterative Method (BIM)	26
2.5.3.	Carlini and Wagner Attack	26
2.5.4.	Projected Gradient Descent	26
2.5.5.	Jacobian saliency map based attack	27
2.5.6.	Countermeasures against adversarial ML attacks	27
2.6.	Parallel computing	28
3.	Tools	29
3.1.	Argus	29
3.1.1.	Argus command	29
3.1.2.	Read Argus command	29
3.2.	NADICS 2.0	29
3.2.1.	Project structure	30
3.2.2.	Preprocessing	30
3.2.3.	Training	30
3.3.	scikit-learn	30
3.4.	Pytorch	32
4.	Dataset preprocessing	33
4.1.	CICAndMal2017	33
4.2.	Secure Water Treatment (SWaT) Dataset	36
5.	NADICS 3 development	38
5.1.	Upgrade to Python 3	38
5.2.	Project structure	38
5.3.	Training tables header	38
5.4.	Label encoding and handling	39
5.5.	Data imbalance handling	39
5.6.	Normalization	39
5.7.	Principal Component Analysis (PCA)	39
5.8.	Feature Selection	40
5.9.	compare module	40
5.10.	Model adapter to PyTorch	40
5.10.1.	Deep Neural Network	40

5.10.2. FGSM and BIM attacks	42
5.10.3. Counter-measurements	42
6. ML Experiments	44
6.1. NADICS configuration	44
6.2. CICandMAL2017	45
6.2.1. NADICS Preprocessing	45
6.2.2. Classical ML	48
6.2.3. Deep Neuronal Network	48
6.3. Secure Water Treatment (SWaT) Dataset	50
6.3.1. NADICS Preprocessing	50
6.3.2. Classical ML	50
6.3.3. Deep Neuronal Network	50
6.3.4. Adversarial attack and training	53
7. Summary	54
7.1. Dataset preprocessing	54
7.2. NADICS 3 development	54
7.3. ML Experiments	55
A. Feature tables	56
B. CICandMAL2017 unexpected feature similarities	59
C. Data splitting	61
List of Figures	62
List of Tables	64
Acronyms	65
Bibliography	67

1. Introduction

1.1. Overview

Automation of processes which are not trivial to define in the form of an algorithms are part of the main technological focus today. The machine learning algorithms are thriving in high availability of training data and computational power, more than ever before. The variety of use cases for ANNs contains finance, transportation, education, natural language processing and medical applications. In this areas ML experienced great improvements in the recent years. After the XOR problem was solved the most recent ML winter was over. This was followed by many great ideas which led to significant advantages in deep learning. (AlphaGO, Convolutional nets etc). The rapidly increasing computational power is driving this methods further.

While improving those methods the focus lies mostly on (true positive rate, accuracy, etc) while ignoring the robustness [1]. In many cases the performance of the network is satisfactory this way. The robustness becomes a problem if the classifier is deployed in a security or safety environment. One of the most popular examples of lacking robustness, was the Tesla accident during which the ANN classified a white truck as a part of the sky following a crash in the same [2].

Adversarial ML gained focus since it's discovery and is now a fast-expanding problem area which is highly interesting to industry and academia [1].

1.2. Why use an IDS

One of the greatest challenges in the modern computerized world is the detection of unwanted behaviours or unwanted activities in a digital infrastructure. An Intrusion Detection System (IDS) is a traditional approach which serves the purpose of reducing the amount of such activities. This work focuses on network environments.

In a network security scenario the firewalls are the first choice. While providing basic security functionality like restriction of access based on protocol, port and Internet Protocol (IP) address range those systems usually not operate on higher Open Systems Interconnection model (OSI model) levels. There are some exceptions like web firewalls. A different approach is a NIDS. It monitors the network traffic and usually detects discrepancies to normal network behaviour which indicates a potential attack or malfunction. Traditionally NIDS are based on rules which are based on knowledge which is provided by a security professional. Because of this, those rules cant cover novel attack patterns and enable a potential thread to a network.

This is a possible use case for a ML agent which due to its generalization properties, has the potential to generalize over never foreseen attack patterns by being trained on known ones. [3]

1.3. Why utilize Machine Learning

Since the variety of attacks and the corresponding malware increases in quantity and quality the traditional rule based NIDS approach will become more obsolete. While the threats are increasing so does the data which is available. This makes the NIDS problem suitable for the ML approach. A ML solution would eliminate the need of a security professional and could be retrained when facing a new threat.

1.4. Why protection against adversarial attacks

In every security critical ML environment the adversarial attacks are one of the greatest threats. This is especially the case if there is an asset which has to be protected by the system. Multiple recent publications warn about the threat which originates from adversarial examples in a NIDS [4], [5], [6], [1] and [7]. In the term of security the question arises how such systems can be compromised. Here one differentiates between benign and adversarial data. An adversarial example is usually an intentionally crafted, for the human eye invisible perturbation of an input sample which has significant impact on the loss function and so on the classification abilities of a given ML algorithm [1]. Usually the adversarial examples are designed as a maximization problem of the prediction error. But there are also adversarial scenarios in which the attacker minimizes the loss function towards a certain example to control the misclassification.

1.5. Research questions

The two fields of Adversarial ML and a ML based NIDS are highly relevant at the time this work is written and already gained some attention due to the risks they imply in combination [5] [4]. For this reason the goal for this thesis is defined as the extension of the NADICS toolset by a metric for adversarial ML. With that a framework which is based on current ML standards will be provided to perform further research in this area.

In order to protect an DNN based classifier against adversarial attacks it is crucial to implement a standardized system to perform training attack and counter measurements against adversarial attacks. This is the final goal of this thesis. Further milestones are defined in table 1.1

1	Providing a preprocessing pipeline for chosen datasets
2	Extending NADICS with scope specific functionalities
3	Train a DNN classifier with NADICS
4	implement an adversarial attack and protections against such

Table 1.1.: Thesis goals

1.6. Structure

This thesis is divided in seven main chapters. After the introductions the next chapter will summarize the conceptual literature review. Here the used methods and theoretical background will be stated and explained. In the following chapter 3 the most used tools which were employed to conduct this work will be introduced and described. This includes all programs and toolkits.

The following three chapters are the task sections. These describe the development and execution of the research done in this thesis. The chapter 4 summarizes the steps taken to reach goal 1 from the table 1.1. It contains the process behind the dataset selection and the required implementations to preprocess them to be used in NADICS.

Goal 2 is achieved in chapter 5. Here all implementations of novel functionalities are described. Those include preprocessing steps but also the DNN implementation. Also included is a description how to utilize those improvements either from the Command-line Interface (CLI) or through configuration files.

The final two goals are thematized in the chapter 6. To create a reference for the performance to the DNN classical ML models like in [4] and [5] were utilized. This is required particularly for the CICandMAL2017 dataset in order to evaluate the different configurations. The classical ML algorithms will not be utilized as an additional indicator for detection of adversarial examples like done in [4] and [5]. Instead the Adversarial Training approach is used to harden the DNN like described in [1].

2. Background

This chapter summarizes the literature research was performed during the work on this thesis. It consists of the theoretical background which was used to target the thesis goals and shaped the methodology which was used to reach said goals. It begins by looking at the task of securing different networks followed by a dive into the topic of ML.

2.1. Networking

A computer network usually consists of two or more hosts which exchange information either over wire or ether. In terms of security the most commonly used classification terms are MAIL and BENIGN. Where MAIL is the label for all malicious activity and BENIGN for normal traffic which is allowed.

2.1.1. Concept

The state of the art conceptional view on a network is the OSI model which is commonly represented in a hierarchy shown in table 2.1. This concept which most wide use is in the World Area Network (WAN) divides every communication in hierarchical parts which have different responsibilities which, when summed up provide a stable connection between two applications. **Layers 7 and 6** ensure that the payload of the communication can be interpreted by the host system regardless of the underlying Operating System (OS) or program version. This is ensured by providing the payload of the communication in a standardized manner. **Layer 5** ensures that every communication line between sub instances of programs is separable by those. This is usually ensured by providing a session key. **Layer 4** is responsible of connecting applications. This is ensured by providing ports per application which can be either used to initialize communication with the other host or providing a service which can be reached by other hosts. In **layer 3** the endpoint communication is handled. One of the main challenges at this point is to bring the payload transitively through a network of hosts. This is managed with identification of each host with an IP address. The underlying complexity of communication between hosts is handled in **layer 2**. The configuration in which hosts are connected to each other without any active components or other hosts between them is called a collision domain. Here every host can communicate with it's neighbors. At this level hosts identify each other with Media Access Controls (MACs). Finally the **layer 1** handles the problem of transmitting the information through the communication medium often referred to as **Layer 0**. [8].

id	Name	responsibility
7	Application layer	Interface methods
6	Presentation layer	formatting of the Payload
5	Session layer	Managing sessions
4	Transport layer	Application connections
3	Network layer	end-to-end connection
2	Data link layer	host-to-host connection
1	Physical layer	data transmission

Table 2.1.: OSI model

2.1.2. Transport

Measured by connection amount the most used transport protocol observed in WAN is Transmission Control Protocol (TCP) followed by User Datagram Protocol (UDP) [9]. These protocols are located on layer 4 of the OSI model. The main difference between them is the connectivity dependence. While TCP focuses on providing a safe connection UDP is a lightweight alternative which can be used to send fast asynchronous messages. This leads to different trade-offs making the decision between these two protocols dependent on the use case.

The sequence diagram in figure 2.1 represents the typical communication during a TCP connection. First the connection has to be initialized. This is represented by the first communication block in the figure and is called 3-way handshake. The data can be transmitted after the initialization. Not shown in the figure is the keep alive signals which are exchanged if there was no data send during a long period of time. The connection can be terminated by performing a 4-way handshake or not responding to keep alive signals. In TCP keepalive is optional. In contrast a UDP communication consists only of one datagram send from one host to another. This ensures fast lightweight communication in the favor of certainty of correct transmission. [8]

2.1.3. Internet protocol

The IP provides so called IP addresses to identify hosts in a network. Those consist of a 32 bit for IPv4 and 128 bit for IPv6. The numbers are usually divided in octets represented in the decimal system and separated by dots to make them human readable. The corresponding representation of IPv6 consists of two octets in hexadecimal system separated by colons. Those addresses can be separated in network ranges by defining the number of most significant bits which identify the networks. Hosts are then identified by the least significant bits. [8]

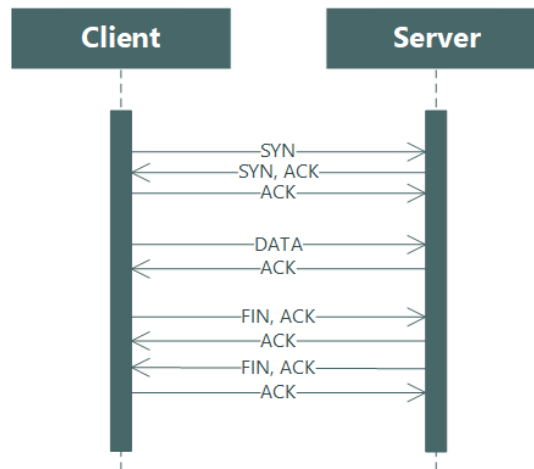


Figure 2.1.: Tcp connection sequence diagram

2.1.4. Firewalls

The increasing connectivity of those networks makes the monitoring of the same a necessity. One of the traditional ways of securing networks is to utilize firewalls. Those can be placed between networks for the favor of restrict access on layers 3 and 4. There are also application firewalls also called reverse proxies which operate on higher OSI model levels, which are outside of the scope of this thesis. A typical application scenario for firewalls consists of multiple networks which represent different departments in an organization and the WAN. The firewall acts as a check point between those networks while enforcing rule based policies. One of the easiest ways of restricting access is by blocking specific hosts or a host range from accessing certain IP addresses or ports. However this policy can be avoided by simply using different hosts or ports. [8]

2.1.5. Intrusion detection systems

Intrusion detection systems can be best described as the alarm systems for a given critical resource. The goal is to notify a responsible person or system if the security of the observed system has been breached. In contrast to an Intrusion Prevention System (IPS) the IDS does not have write access to the critical resource making it's purpose dependent on external incident handling. This ensures the resilience of said resource against false positive based actions of the IDS. This happens at the cost to longer attack reaction times due to the incident handling by external actors like humans. [10]

The main goal of an NIDS is the detect malicious activities inside a critical network. It is usually an appliance, a group of appliances or software which has access to the networking

2. Background

anomaly	self-learning	non time series	rule modelling
			descriptive statistics
	programmed	time series	ANN
		descriptive stat	simple stat
			simple rule-based
signature	programmed		threshold
		default deny	state series modelling
		state-modelling	state-transition
			petri-net
		expert-system	NIDES A.14, EMERALD A.19, MIDAS-direct A.2, DIDS A.9, MIDAS(2) A.2
signature inspired	self-learning	string-matching	NSM A.6
		simple rule-based	NADIR A.7, NADIR(2) A.7, ASAX A.10, Bro A.20, JiNao A.18, Haystack(2) A.1
		automatic feature sel	Ripper A.21

Table 2.2.: Classification of detection principles. Source: [10]

traffic bypassing switch and Virtual LAN (VLAN) configurations with the utilization of SPAN or TAP interfaces. There are various open source and enterprise implementations. One of the most famous ones in the open source community is SNORT. There are two approaches for detecting malicious traffic which are practiced frequently, which are described in the chapters 2.1.5 and 2.1.6.

The performance of an IDS can be measured in a confusion matrix. Since false positive and negative rates would lead to less trust to the system by the user, those should be minimized. On the other hand True positive/negative values are maximized.

Policy definition

There are two approaches in deriving of a policy for a IDS. The most intuitive way is to program the IDS based on rules. Those are usually defined by professionals or other systems. This method is dependent on the professionals knowledge and is vulnerable to human bias [11]. A self learning algorithm can be trained based on a reference dataset or through continuous improvement based on the data the system faces. [10]

While training a ML algorithm one should be aware of the usually not equally distributed relative quantities of BENIGN and MAIL samples in the training dataset. All metrics in 2.3 do not perform well on imbalanced class distributions.

Anomaly detectors

Anomaly detection in terms of IDSs is a long established methodology which is constantly improved for more than 40 years [11]. Table 2.2 shows a categorical classification of IDSs. Anomaly based IDS can be further categorized based on their rule definition process.

In the **programmed** class a professional has to define rules for normal or abnormal behaviour of the network. This can be done with Deceptive statistics in which a profile of normal

statistical behaviour is defined. This can be done with simple statistics where the decision about the normal behaviour is derived from higher level components, with simple rule-based statistics where the user provides rules to control statistic specific behaviour of the system, or Thresholds in which the user has the possibility to define normal behaviour in terms of statistical borders. The default deny IDS explicitly defines the scope of benign behaviour for a given system. All deviations from said behaviour are considered as an intrusion. This can be done by defining a state machine with nodes which represent all possible states of the machine and transitions which represent normal actions. Any action that can't be described by the state machine is then considered abnormal. [10] This makes the system robust against unusually behaviour, however it can only be used in constant environments like a production line of Programmable Logic Controller (PLC) systems.

Self-learning anomaly detection IDSs employ a ML model to detect anomalies. This systems learn by observing traffic over a period of time which is then used as a reference to train the algorithm. Here the main discriminator is whether time series is used or not. When time series are not considered the system is either programmed to model rules which describe a normal behaviour of the critical resource or does the same in forms of statistics. If time series is considered the complexity of the behavioural patterns and the model increases. An ANN can be used to model this behaviour as a black box. [10]

2.1.6. Signature detectors

A signature of an attack or unwanted behaviour can be derived based on knowledge about the model of the intrusive process. In the ideal case those systems are not affected by the form of the normal background traffic. Here [10] only defines programmed systems which can be implemented on the basis of a state machine like a petri net, an expert system, rules or string matching. [10]

2.1.7. Compound detectors

This systems form a compound of signature detectors which focus on identification of abnormal behaviour and anomaly detectors which identify normal behaviour and notify the user if it is not given. In table 2.2 those are called "signature inspired". These detectors have better classification properties between different classes of behaviour observed in the system. This is due the fact that they focus on identifying different classes of behaviour. This way the different classes can be related to each other which makes it easier to identify relevant features and discriminate them better in the feature space with ML algorithms introduced in chapter 2.3. [10]

2.1.8. Counter measurements

Since the detection by a NIDS means lower exploitation potential, the malware designers came up with new ideas to avoid those. Many of the counter measurements are build around code permutation or tunnelling. This is due the fact that most NIDS are static rule based.

2.2. Machine Learning

Machine learning describes the goal for a computer or other kind of machine to imitate the learning behaviour of a cognitive being like the human. This ability is not limited to simulating the actual wetware which enables the learning for biological organisms.

2.2.1. Supervision

Any ML scenario is typically defined in one of three classes which define the approach to the problem. They are usually dependent on the quantity of labeled data or personnel to do so and the separability of the dataset. [11] **Supervised learning** This is the most convenient scenario since here one have the dataset with the correct labels. for the corresponding classes. The task for the ML agent is to learn the relation between the features in the dataset and the labels. The algorithm is presented with the pairings and the final performance is measured based on the performance on unseen data. [12] **Semi-supervised learning** The dataset here is only containing some labels or only a specific class is pre labeled. For example a cluster of dataset tuples has only one labeled tuple in this case the algorithm would label all the other tuples with this label. [12] **unsupervised learning** There is no labeled training set available or no training set at all. The algorithms task is to find different groups or clusters which would separate the dataset. [12]

2.2.2. Reinforcement Learning (RL)

Reinforcement Learning implies an agent which learns by directly interacting with its environment. The goal is the creation of a good action policy driven by the maximization of a reward function. The world which the agent interacts with is defined as a Markov decision process (MDP). A MDP is by definition a tuple (S, A, T, r, δ) with the following definitions. [12]

S	Finite set of states
A	Finite set of all possible actions
T	Set of transition probabilities
r	Reward function
δ	Discount factor

There are two techniques which are widely applied in the scope of RL. In Q-Learning

2.2.3. Regression and classification

Linear regression is given when the Inference model has to predict continuous numbers rather than distinct classes. While in a classification problem the labels are discrete elements from a set of classes.

Dataset

Unlike in real time training models like reinforcement learning (2.2.2) the inference ML is based on a fixed dataset. This is usually a matrix X which consists of data points x in the feature hyperspace. Features are the attributes that a data point has. The desired corresponding target values which have to be inferred are presented in the vector y . The estimator $f_{\omega}(x_i)$ which predicts the labels $\hat{y}$ usually takes the outcome of the learning process called the weights as a argument. Weights are represented with ω . [12]

Loss Function

Usually an inference ML task is described as a minimisation problem. The goal of this is the Loss or error function. Equation 2.1 shows the for regression tasks typical loss function Least Squares

$$E_{LS}(\omega) = \frac{1}{2} \sum_{i=1}^N (f_{\omega}(x_i) - y_i)^2 \quad (2.1)$$

Training

In order to ensure that the agent performs well on unseen data the dataset is usually divided in multiple fractions. Those are:

Training dataset: Is used to actually train the agent. The ML algorithm has access to the data and the corresponding labels

Validation dataset: Is the dataset which is used to track the learning performance of the agent. It's feedback can be used to improve training but the labels cannot be used directly. Exceptions here are i.e. k-fold clustering.

Test set: This dataset is used to evaluate the performance of the agent at the end. The agent is not allowed to see any portion of this data before it is finished with the training process.

Usually one plots the development of the training and validation predictions over time in a learn curve. This method can be very helpful in identifying problems during learning like overfitting.

2.2.4. Overfitting

One speaks of overfitting when the ML agent is too dependent on the training dataset. This usually means, that the agent is performing very poor on unseen data because the model fits training data perfectly. One can detect overfitting by observing the learning curve. If the predictions are always correct for the training dataset and are poor for the validation dataset, it's a strong indicator for overfitting. [13]

The most widely used method to prevent overfitting is regularisation. Regularisation adds noise to the dataset to make it harder for the agent to train on it directly. This results in slight

2. Background

Accuracy	$acc = \frac{TP+TN}{TP+TN+FP+FN}$
Precision	$prec = \frac{TP}{TP+FP}$
Sensitivity/Recall	$rec = \frac{TP}{TP+FN}$
Specificity	$TNR = \frac{TN}{FP+TN}$
False Negative Rate	$FNR = \frac{FN}{TP+FN}$
False Positive Rate	$FPR = \frac{FP}{FP+TN}$
F1 Score	$f1 = \frac{2*prec*rec}{prec+rec}$

Table 2.3.: Caption

perturbations of the samples with each learning step leading to a more generalizing agent. [13]

2.2.5. Performance metrics

In the industry and research there are established metrics to measure the performance of an ML model in terms of accuracy and performance. Here we use the terms from the confusion table.

Predicted	True condition	
	$y = 1$	$y = 0$
$y = 1$	True Positive (TP)	False Positive (FP)
$y = 0$	False Negative (FN)	True Negative (TN)

Where True Positive and True Negative are the correct predictions. The table 2.3 illustrates the most important ones which are considered as prior art in our work. One should take in consideration that the above metrics work best on balanced classes and can be very imprecise otherwise.

A Receiver Operating Characteristic curve (ROC curve) is a method to illustrate the diagnostic performance of a binary classifier in dependence of the discrimination setting. The ROC curve is created by plotting the False Positive Rate (FPR) in the x axis of a cartesian coordinate system and the Recall in the corresponding y axis. Recall is also called True Positive Rate (TPR) [12]. Figure 2.2 illustrates the dependencies in a ROC curve scenario.

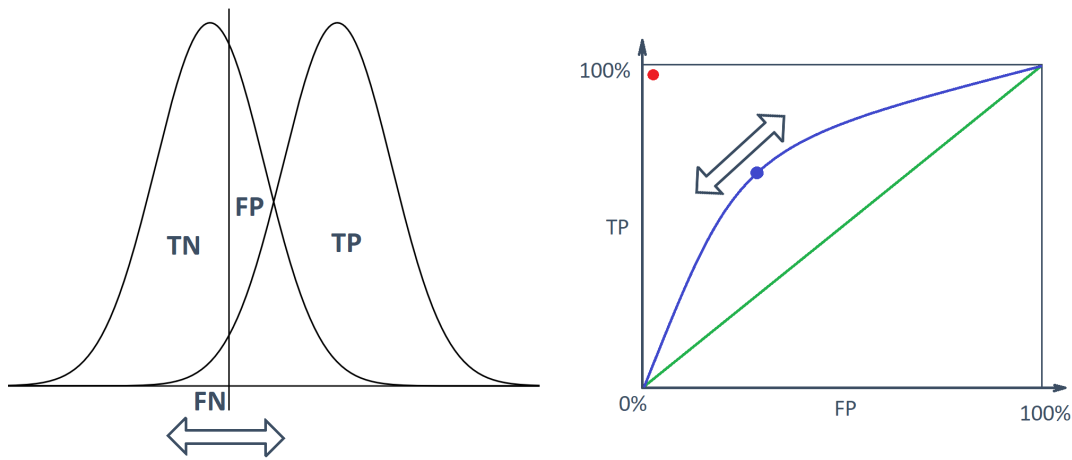


Figure 2.2.: Left figure shows the distribution of samples for one feature in a binary classification scenario. The vertical line is the discriminator which classifies every sample on its left as negative and positive on the other side. The right image shows an example of an ROC curve. While the green line represents a random classifier and the blue the classifier resulting from the left figure. The red point represents an optimal classifier with 100% TP with 0% FP rate. By moving the discriminator function along the x axis on the left image the ratio between True Negative (TN), False Negative (FN), False Positive (FP) and FP changes which results in a movement on the blue ROC curve (right). The goal is to find a suitable trade-off between two axes.

2.2.6. K fold cross validation

A popular method to increase the amount of available data during training is the K-fold cross validation. The idea is to split the training data into K folds and retrain the model leaving out one fold at the time, using the leaved out fold as the validation dataset. [12]

2.2.7. No free lunch theorem

In the world of machine learning one assumes that there is no perfect algorithm for all learning scenarios. One has always to make trade offs between different ML algorithms and their configurations. As a consequence of this a variety of algorithms needs to be developed to much as many leaning scenarios as possible. [12]

2.2.8. Feature Selection

Filter methods

Values with a relatively low variance do not contribute much to final classification. This is the case when variance is as high as the noise of the feature rendering it useless or when the feature has the same value for every tuple in the dataset.

One additional possibility to filter out low contributing features is to measure covariance with the label and with other features. In this scope the Pearson Correlation is very common which is defined with the equation 2.2 [14]. The criteria to filter the features out is based on the following two assumptions.

High/negative correlation with other features: In this case the assumption is that the two features describe the same property of the situation in different ways.

Zero correlation to the label: This indicates that the classification one is looking for is not dependent on the feature. However It could be that the feature can still contribute to the classification process in combination with other features.

$$r = \frac{\Sigma(x - \bar{x})(y - \bar{y})}{\sqrt{\Sigma(x - \bar{x})^2 \Sigma(y - \bar{y})^2}} \quad (2.2)$$

wrapper methods

This category of methods considers the selection of features as a search problem. One method is Recursive Feature Elimination (RFE). It eliminates features by removing a subset recursively. It does it by recursively considering smaller feature sets. The model is trained on the initial dataset and feature importances are derived based on their individual performance. Then the classifier is trained on less features until the quantity of desired features is reached. [14]

2.2.9. Principal Component Analysis

Principal Component Analysis (PCA) is a method for dimensionality reduction like feature selection but it focuses on extracting the meaning of the features and projecting them in a meaningful matter. The goal is to project a highly dimensional feature space onto less features. This is achieved by deriving a multidimensional linear plane in the feature space on which the data is located. This implies the assumption that the data is not equally distributed in the original feature space. [13]

2.3. Classical ML Algorithms

All non neuron based machine learning approaches are called classical in the scope of this thesis. One should not confuse this terminology with outdated nor should one think that neuron based models are novel. The following algorithms are still in use today in many various applications.

2.3.1. Gaussian Naive Bayes

This pure probabilistic approach is based on the Bayesian inference. Which is based on the maximization of the posterior distribution (Equation 2.3), where $p(\mathbf{y}|\mathbf{X}, \beta, \cdot)$ is the likelihood, $p(\mathbf{w}|\cdot)$ the prior and $p(\mathbf{y}|\mathbf{X}, \beta, \cdot)$ the normalizing constant. The prior is not dependent on the actual data and is a possibility to bring expert knowledge into the model. The likelihood is the probability that the data point belongs to class y given the data x . Since the normalization has no effect on the predictions it is usually not used (Equation 2.4). [12]

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X}) * p(\mathbf{w}|\cdot)}{p(\mathbf{y}|\mathbf{X})} \quad (2.3)$$

$$\propto p(\mathbf{y}|\mathbf{X}) * p(\mathbf{w}) \quad (2.4)$$

If one maximizes the posterior distribution from 2.3 one gets the Maximum A Posteriori (MAP) hypothesis.

The naive Bayes classification algorithm has its name because of the simplified calculation methods for the probabilities within. The features of the probability $p(\mathbf{d}|\mathbf{h})$ are assumed to be conditionally independent and are calculated like in the equation 2.5. [12]

$$p(\mathbf{d}|\mathbf{h}) = p(d_1) * p(d_2) * \dots \quad (2.5)$$

This appears very counter intuitive since the real word data features are usually dependent, however this method performs surprisingly well even on co dependent features. In the training the naive Bayes is very fast due the fact that only the probability for every class and the probability for every class given the data has to be calculated.

class distribution: Here one has to calculate the relative probability that a specific class

occurs in general. This is a Maximum Likelihood Estimation (MLE) with the equation 2.6
class probability given data: This values gives the conditional probability for a feature given a specific class. The determination is given in the equation 2.7

$$p_i(y_i = 0) = \frac{|\{y_i \in \mathbf{y} | y_i = 0\}|}{|\mathbf{y}|} \quad (2.6)$$

$$p_i(x_i = 0 | y = 0) = \frac{|\{x \in \mathbf{X}_i | x = 0 \ \& \ y_x = 0\}|}{|\{y \in \mathbf{Y} | y = 0\}|} \quad (2.7)$$

Predict data points: The predictions are made with a MAP like in equation 2.4.

To extend the naive Bayes to a real world scenario, it can be modeled as the Gaussian naive Bayes, which is the approach one usually used in data science. The predictions of this method are calculated with the Gaussian probability density function (PDF). In case of two classes this distribution is a Bernoulli naive Bayes. [12]

2.3.2. K-means Clustering

Clustering problems in general is an example for processing of unlabeled data $\mathbf{X}$ and are used to discover latent structure in it by detecting dissimilarities. The similarity function $s(\cdot, \cdot) : \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$ gives the similarity for two points in data space. The value of the similarity function increases with the similarity of those points. Alternatively one can use dissimilarity functions which are distance functions like Euclidean distance or Mahalanobis distance over the feature hyper space. [13]

In the K-means algorithm one measures dissimilarity with a distance measurement function over the feature space $\mu_k \in \mathbb{R}^D$. Where D is the dimensionality. Every cluster is defined by its centroid which indicates the corresponding class in an one hot encoding manner $z_i \in \{0, 1\}^K$. The measurement which indicates the performance of the clustering is called the K-means objective or distortion measure, Equation 2.8. [12]

$$J(\mathbf{X}, \mathbf{Z}, \mu) = \sum_{i=1}^N \sum_{k=1}^K z_{ik} \|x_i - \mu_k\|_2^2 \quad (2.8)$$

$$\mathbf{Z}^*, \mu^* = \arg \min_{\mathbf{Z}, \mu} J(\mathbf{X}, \mathbf{Z}, \mu) \quad (2.9)$$

The goal is to find the best centroids μ and cluster the assignments $\mathbf{Z}$ (Equation 2.9). The both hyperparameters $\mathbf{Z}$ and μ are then optimized in alternating manner resulting in Lloyd's algorithm.

1. $\mu = \{\mu_1, \dots, \mu_K\}$ # Initialize centroids
2. Solve $\min_{\mathbf{Z}} J(\mathbf{X}, \mathbf{Z}, \mu)$
3. Solve $\min_{\mu} J(\mathbf{X}, \mathbf{Z}, \mu)$
4. If objective $J(\mathbf{X}, \mathbf{Z}, \mu)$ has not converged, goto 2.

Random initializations of the centroids can however be very problematic and lead to not optimal solutions after training. This is why the initialization is usually done with the

K-means++ algorithm. In this approach firstly one centroid is chosen at random, then the distance is calculated to every point and the farthest point is chosen as the next centroid. This procedure is repeated until all centroids are chosen. [13]

2.3.3. Tree based models

Decision trees is a widely used ML method which work by partitioning the hyperspace in cuboid regions. By definition a decision tree is a binary one with each node separating only one feature. The process of classifying an arbitrary data point consists of traversing the binary tree with respect to the data points features. A leaf node corresponds to a specific class. Figure 2.3 Illustrates a typical decision tree with the corresponding partitioned feature space. The partitioning is determined by the parameters θ_1 and θ_2 which are determined in the learning process. [12]

The loss function used in decision trees is usually the impurity measure i , which is higher when classes are equally distributed in the partition. Equations 2.10, 2.11 and 2.12 show the formulas for the most common used impurity measures: Misclassification rate, Entropy and Gini index. In order to determine where to split one calculates every possibility for a split in the form of a grid search and picks the θ with the highest difference in the impurity measure. [12]

$$i_E(t) = 1 - \max_c \pi_c \quad (2.10)$$

$$i_H(t) = - \sum_{c_i \in C} \pi_{c_i} \log \pi_{c_i} \quad (2.11)$$

$$i_G(t) = \sum_{c_i \in C} \pi_{c_i} (1 - \pi_{c_i}) = 1 - \sum_{c_i \in C} \pi_{c_i}^2 \quad (2.12)$$

There are multiple stopping criterion configurations the most common are the stopping when the distribution in the branch is pure, a maximum depth is reached, there are too few samples in the branch, the benefit is below a specific threshold or based on accuracy. It is also a common practice to prune the tree after training. This is done by removing an arbitrary subtree which yields the highest improvement with respect to the validation set. [13] [12]

Random Forrest

The random forest is like its name suggests an ensemble of multiple decision trees, which act independently from each other. This method is based on the concept of the wisdom of crowds. Every tree gets a vote which is considered in the final classification. The class with the most votes is the predicted class.

To ensure that the decision trees are diverse there are multiple approaches:

Bagging: Every ML model is dependent on the training data it gets. This is also true for

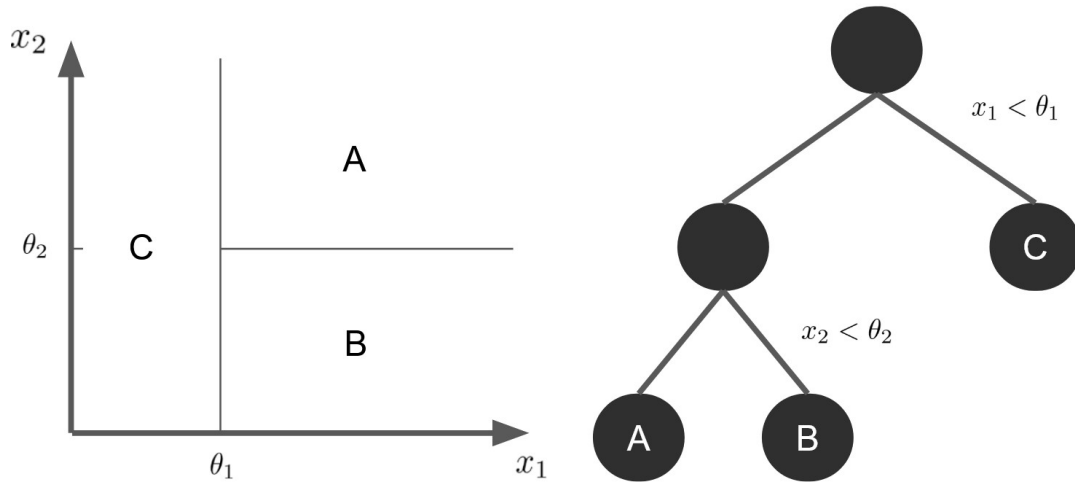


Figure 2.3.: (Left) A possible partition of the feature space with two features. the constants θ_1 and θ_2 are the learned weights of the classifier. (Right) the corresponding possible decision tree model.

decision trees. Here every tree receives a non exclusive subset of the original data. Those sub sets should not be confused with batches.

Feature Randomness: This widely used method divides the features of the training dataset into non exclusive feature sub sets which contain all the training tuples. So every decision tree has access to a unique set of features. [12]

2.3.4. AdaBoost

Boosting is very power full especially when applied on base classifiers or weak learners, which perform only slightly better than random. The initial Idea was firstly documented in [15]. The first success full boosting algorithm is Adaptive Boosting or AdaBoost. This algorithm are based on decision tree learners, with one fork. AdaBoost works with weights on data points. More difficult observations get more weight. During learning new learners are added and are put on high weighted data points, ensuring to have more learners on hard problems.

the prediction is based on a voting system of the learners. More accurate learners are weighted higher, ensuring that their vote has a greater impact on the final result. [13]

2.3.5. Gradient boosting

The generalization of AdaBoost is called Gradient boosting and is based on [16] algorithm. Gradient boosting requires the same elements as the AdaBoost, namely loss function, weak learners and a learner adding mechanism. **Loss Function:** It has to be differentiable

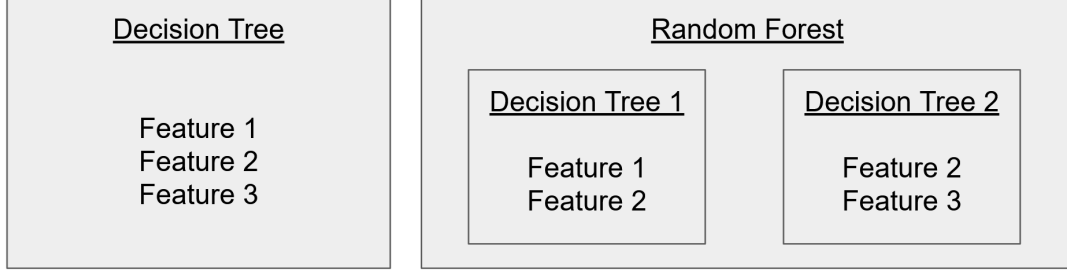


Figure 2.4.: To an omniscient decision tree (left) all features are available. In a random forest with feature randomness, each tree gets an unique feature subset. This ensures that the trained trees are also different, which contributes to the wisdom of the crowd effect.

Weak Learner: here also decision trees are used. The trees are constructed in a greedy manner with impurity measurements like stated in equations 2.11 and 2.12. In contrast to AdaBoost the trees can be deeper than one split.

Additive Model: Trees are added one by one with the learning iterations. Already existing trees are not changed by the process. Usually the gradient descent is used to minimize the loss by adding trees. [12]

2.3.6. k-Nearest Neighbors

This ML technique is based on the assumption that data points which have a close proximity to each other in the feature hyper space are also strongly related in terms of their classification. Basically if there is a new point to classify it is analyzed in terms of distance to other points and is classified based on this. The hyper parameter k from the name, corresponds to the quantity of the closest neighbors to consider. Figure 2.5 shows the example for an k-NN classification feature space. [12]

$$p(y = c | \mathbf{x}, k) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}(y_i = c) \quad (2.13)$$

$$\hat{y} = \arg \max_c p(y = c | \mathbf{x}, k) \quad (2.14)$$

Equation 2.13 describes the classification process for a given data point mathematically while equation 2.14 describes the prediction of the classifier.

The typical distance measurements for k-NN classifiers are L_1 norm, L_2 norm (Euclidean distance), L_∞ and the Mahalanobis distance etc.

On some distance measurements there are problems with the scaling of the dataset. if feature x_1 has the range $[0, 10]$ and feature x_2 the range $[0, 0.1]$ the Euclidean distance does not perform well. Typically one can standardize the data at this point. An alternative approach is

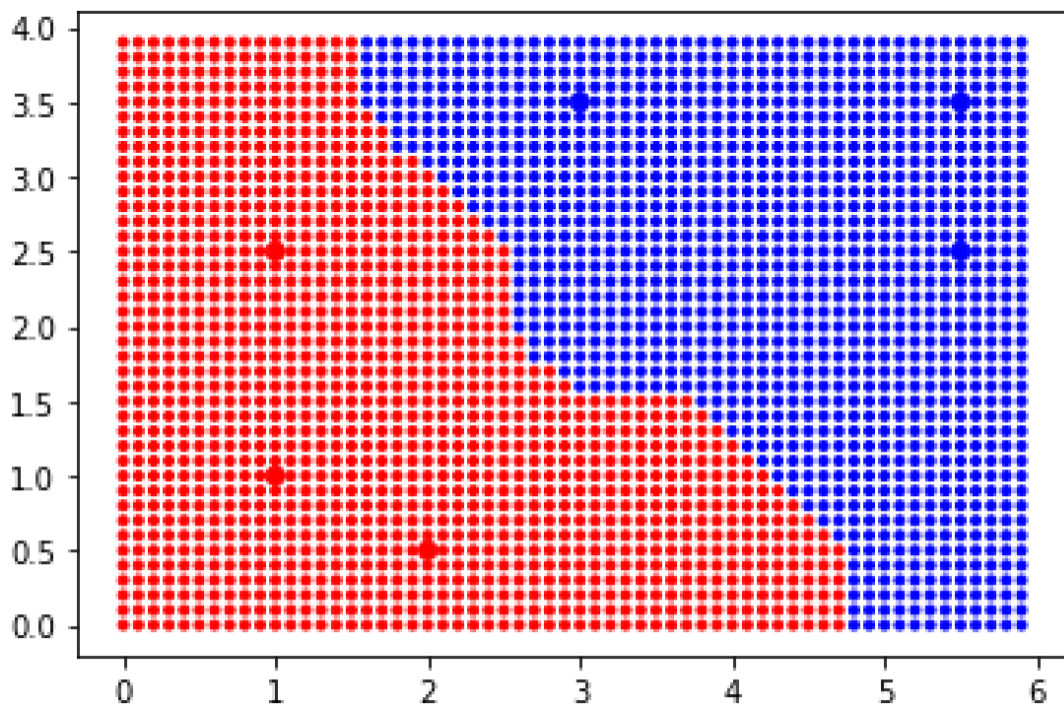


Figure 2.5.: An example for a k-NN hyperspace with 6 data points, two features and two classes. One can see that the hyperspace can be separated in two partition fairly easy with only few data points.

to use a distance measurement which is by definition not sensitive to scaling differences like the Mahalanobis distance (equation 2.15)

$$mahalanobis(x_1, x_2) = \sqrt{(x_1 - x_2)^T \Sigma^{-1} (x_1 - x_2)} \quad (2.15)$$

The k-NN classifier is an example for a ML model which does not require a training phase. However the data has to be present and will be processed for every classification. This is very inefficient for large datasets, however this classifier is very precise and also generalizes well, for classification problems with few features and data points. [12]

2.3.7. Support Vector Machines (SVM)

Support vector machines (SVMs) are an example of supervised learning. The classification is based on one or multiple hyperplanes in the feature space. This method can be used for classification and regression. The training of a Support Vector Machine (SVM) consists in finding a hyperplane which separates the different classes of the dataset and them maximize the distance to those. There are multiple configurations for SVMs with more than one hyper plane. [12]

2.4. Artificial Neural Networks

ANNs gained a lot of attention recently and are implemented widely in state of the art technologies like industry 4.0, autonomous driving and medical applications. ANNs use the principle of emergence to simulate learning behaviour. This is a common property to a Spiking Neural Network (SNN) which can be found in the brains of mammals animals. A core property of ML algorithms is the ability to generalize. By that one means the ability to extract class specific features and with this knowledge identify never seen data.

The neuron

The perceptron algorithm is often introduced as the first implementation of a neuron in the ANN style. This algorithm is a linear discriminant model and can be used for linear separation in the feature space. The algorithm follows the equation 2.16. The vector $\phi(x)$ usually also features a bias component. The function $f(\cdot)$ is called an activation function and is defined with the equation 2.17 [13]

$$y(x) = f(w^T \phi(X)) \quad (2.16)$$

$$f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases} \quad (2.17)$$

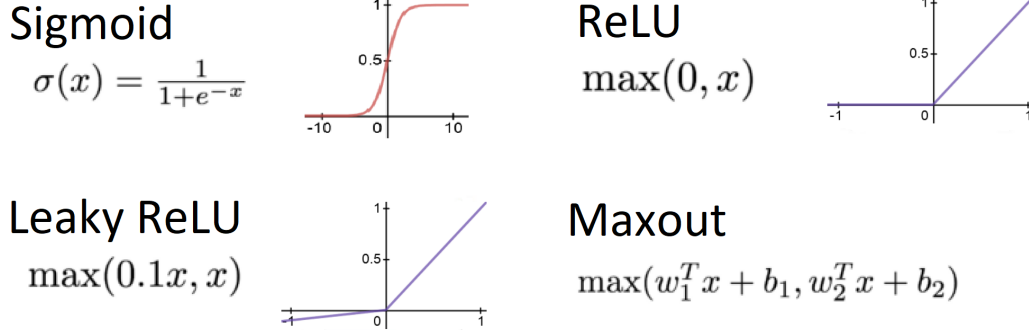


Figure 2.6.: Commonly used activation functions

Multi layer perceptron

The mathematician Seymour Papert proved in 1969 that the single perceptron could not depict the XOR operation. Which then got famous as the XOR problem and led to the AI winter. This problem was later solved with the Multi Layer Perceptron (MLP). By adding a non linear activation function the model can feature multiple layers and the features can be mapped into a space where they are linearly separable. [13]

Feed forward networks

The basic neural network model consists neurons which them self consists of a series of functional transformations. At first the inputs are combined linearly with the equation 2.18. The (1) properties in the equation indicate the id of the network layer. In this case the parameters correspond to the first layer. The quantities a_j are called activations. Those are fed in a differentiable, non linear activation function $h(\cdot)$ which is defined in equation 2.19. [13]

$$a_j = \sigma_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \quad (2.18)$$

$$z_j = h(a_j) \quad (2.19)$$

Activation functions

Figure 2.6 shows the six most common used activation functions in the DNN environment.

$$y_k = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \quad (2.20)$$

The Softmax function is a notable exception because it properties allows it to normalize the output. For this reason it is often the first choice for the output layer in a discrete classification scenario with multiple classes.

Loss function

The metrics for the distance between the correct solution and the predicted one is the loss or cost function. This function is also used for training. The most common function to define loss in a multi class classification scenario in a feed forward network is the Cross entropy loss. The function is defined in equation 2.21.

$$E(W) = -\sum_{n=1}^N \sum_{k=1}^K (y_n k \log f_k(x_n, W)) \quad (2.21)$$

The most common encountered problem during learning results from the non-convex property of $E(W)$. The effect is that the learning algorithm can get stuck in a local minima resulting in a non ideal final learning state.

Backpropagation

The learning process in feed forward neural networks is realized with backpropagation. The defined structure of the feed-forward networks allows the efficient computation of the partial derivative for every weight given the desired output with respect to the input. This can be done by applying the chain rule recursively on the layers.

Regularization

The main objective of regularization is to prevent the model from overfitting. This is especially a problem for models with high capacity, which at some point simply learn the dataset. A simple method to counteract is early stopping in which an algorithm is not trained to the absolute minimum but stopped after a fixed amount of rounds. The two other commonly used methods are L1 and L2 regularization. [13]

Also methods like dropout where some of the neurons are randomly excluded from the network for a certain training period has been proven to be effective.

2.4.1. Dimensionality

Curse of dimensionality

The main motivation behind using only relevant features is the complexity which rises with every additional feature dimension exponentially. This results in more sparse data tuples in the feature space making the training process more difficult. In example a feature space with two dimensions with 10 possible values per feature results in 100 possible combinations. By adding another feature with 10 possible values the number of possible combinations increases to 1000.

Scaling of data space

Scaling is the set of all methodologies to stretch the data distribution across the feature space. This is usually utilized to stretch the whole dataset to a specific range e.g. $[0, 1]$. In this thesis

min-max scaling	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$
Standard scaler	$x' = (x - x_{mean}) / x_{std}$

Table 2.4.: Scaling methods considered in this work with x as the input and x' as the scaled value

we consider the scaling methods introduced in table 2.4.

min-max scaler rescales the distribution of feature data points in such a way that they are in the range of $[0, 1]$. The main issue with this problem is that it scales linearly to this space. That means that an extreme outlier defines the outer border and can lead to compression of all other values to a very narrow space. [17]

StandardScaler scales the data to unit variance and sets the mean to zero. However this approach still does not handle outliers very well and can compress most data-points in their presence. [17]

RobustScaler The advantage of this scaler is it's robustness against a small number of extreme outliers.

Normalization

Normalization and scaling are often used as synonyms. Although they are often performed in the same step, they are not the same. By definition a normalization function changes the distribution function of a given dataset in such a way that it results in a normal distribution also known as the Gaussian bell curve. **PowerTransformer** Is a scaler with transforming properties and utilizes power transformation to generate a Gaussian like distribution. By finding the optimal scaling factor it stabilizes the variance with a MLE.

QuantileTransformer has an additional parameter to define the desired output distribution. Here one can choose either fully Gaussian or uniform distribution.

2.4.2. Data Availability

A successful machine learning algorithm is often highly dependent of the amount of training data. This is especially the case for DNNs. Said data is often not available in the desired quantity or features highly unbalanced between the data point labels. The naive approach to counteract the latest said problem would be to equalize the available data by down-sampling the majorly present classes until they meet the class with the lowest amounts of samples. However this methodology has the potential to sacrifice huge chunks of data. This is especially the case for larger amounts of labels.

This environment is called unbalanced learn in the scikit-learn framework. We will introduce the most used methods in this chapter.

Another naive approach is to resample the training data with replacement. However this would create exact copies of the data points, which could easily lead to overfitting on said

data points.

2.5. Adversarial Machine Learning Attack

With growing application of ML methods it is also important to be aware of the security threads which target it. Like optical illusions which can be experienced by any biological brain [18] there are also samples which can fool machine learning algorithms. Those are called adversarial examples in the ML scenario. The awareness of those examples is crucial as soon as the ML system has to handle impact full tasks like the ones in medicine or cyber security. The field of Adversarial Machine Learning has emerged to systematically analyze the impact of those adversarial examples on ML models.

A variety of soft spots have been documented in [19]. In [20] the authors showed the responsiveness to such examples to input-output trained networks. They also showed that those examples can be reproduced on different sets of the same dataset implying that those are not random disturbances in the learning process and are dependent on the task and image format.

Many of ML algorithms that are used in industry are vulnerable to such attacks. [1]

The National Institute of Standards and Technology (NIST) categorizes the adversarial attack types against ML algorithms in evasion, extraction, poisoning and inference. [4] (needs elaboration)

One can classify the adversarial attack based on the following features:

Location in ML pipeline: Different opportunities arise dependent on the fact if the attacker has access to the network during training or not. If the attacker can "poison" the network, the attack vector is called a training attack and inference attack (evasion attack) otherwise. The main goal of the poisoning attack is to mislead the network in its training phase using adversarial examples. This have a much higher potential impact than the evasion attack since it has an effect on every model deployment after the training phase. The attack can also be used to steal the ML model through a hidden channel in the training process [1].

Adversary's knowledge: A further very important classification paradigm for adversarial attack is the knowledge about the attacked ML algorithm. In literature one defines a black box attack when the ML algorithm is not known. In this case the attacker only has access to the samples from the data hyperspace, the corresponding label and in some cases partial access to the ML model. In a white box scenario one tries to attack a specific ml algorithm or a set of those. Here the attacker can analyze and exploit the algorithm specific properties [19]. Black box attacks are usually divided in query-based and zero-query attacks. A query based attack describes the attack paradigm where the attacker has the role of a user. This query response data is then used to generate an adversarial example. In the zero-query scenario the adversary has no access to the ML model but to a dataset to of test samples. [1]

Adversarial intent specificity: This sub feature of evasion attacks determines the motivation of the attacker and is categorized in targeted and Non-targeted attacks. As the name suggests the non targeted attack focuses on miss classifying a simple dataset like an image which is

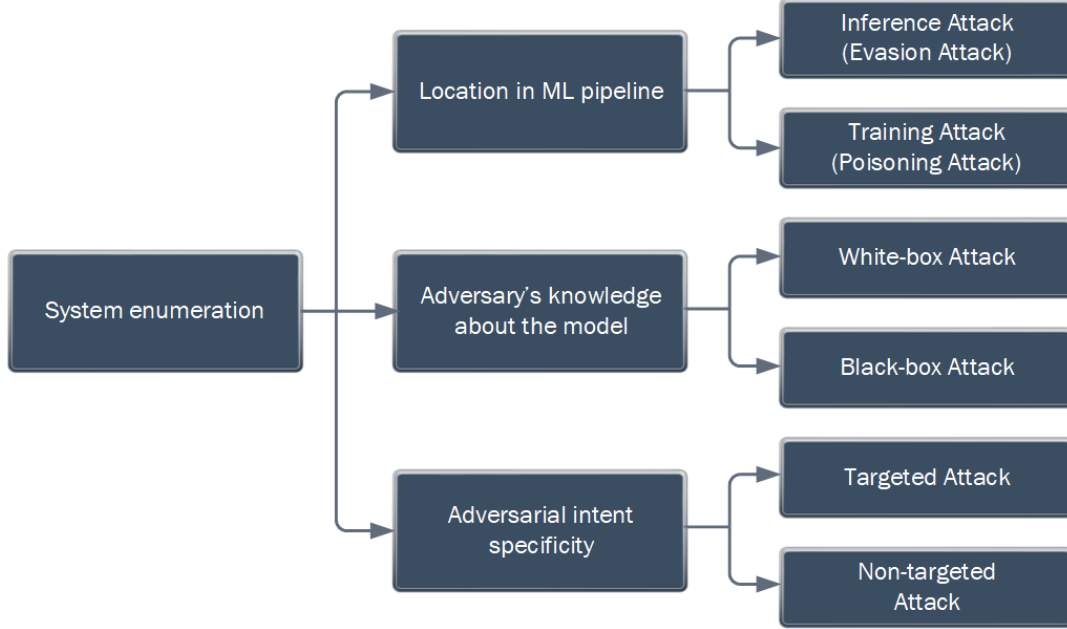


Figure 2.7.: Adversarial attackers perspective after [1]. There are three main features which he has to extract after the enumeration phase. Those are subdivided in different attack approaches which are dependent on the information available to the attacker and his motivation.

then not classified as the trained label. A targeted attack is to modify the source dataset to classify it as a given false label by an arbitrary machine learning algorithm.

There is research in poison attacks which shows that with sufficient resources one can alter data labels and compromise ML models leading to an error rate of up to 100% [21]

Since in ANNs one has very much control over the learning process through gradients there is much potential to find a method to compute such adversarial examples [1]. Due to constantly increasing interest and use of ANNs this forms a serious security thread [19]. The networks in this work are no exception and are therefore also a potential attack vector.

$$x^* = x + \epsilon * \delta \quad (2.22)$$

Equation 2.22 shows a typical adversarial example and is the basis for every adversarial example generation in this work. An adversarial input x^* is created by adding a scaled ϵ perturbation δ to an original label x .

Securing the own ML model against adversarial attacks is challenging because of the asymmetric nature of adversarial attacks. It is generally harder to secure the whole attack area of an ML model than the finding of an adversarial attack vector.

The first documented robustness problem of DNN was discovered by [20]. After which concrete attack methods were developed. The most used ones are introduced below.

2.5.1. fast gradient sign method (FGSM)

Firstly introduced in [22] the fast gradient sign method is one of the widest used algorithms to produce adversarial examples. The method works by using the gradients of the loss by including the input data to find a noise vector which points in the opposite direction of the loss function and maximizes the loss in that way [22].

$$x^* = x + \epsilon * \text{sign}(\nabla_x L(\theta, x, y)) \quad (2.23)$$

A non-targeted arbitrary adversarial example can be created by adding noise to a given input data sample. Equation 2.5.1 defines the process. Where x is the normal input, ϵ is the scaling hyper parameter and L the loss function of the model. Since one is interested in creating an input dataset which maximizes the loss, the method searches for the value change in each dimension to do so. The term "fast" in the name originates from the fact that one can compute the corresponding changes very fast by applying the chain rule to the ML model, which is also used to train it. The question how big the computed change is, does not contribute to the final noise intensity, since it is normalized by the *sign* function. The method can also be used to generate targeted adversarial examples.

2.5.2. Basic Iterative Method (BIM)

After further development of the Fast Gradient Sign Method (FGSM) the basic iterative method was introduced [23]. By scaling the noise vector down and applying the FGSM multiple times one can maximize the loss by reaching the border of the ϵ norm hyper ball.

2.5.3. Carlini and Wagner Attack

In [24] a targeted adversarial attack method was introduced. This approach handles the adversarial attack as a minimization problem which alters the input data in such a way that the loss to a given target label is minimized.

2.5.4. Projected Gradient Descent

In contrast to the normal gradient descent method the projected gradient descent minimizes the gradient function in subject to a constraint. This constraint could be the false label for a given dataset. With this approach [25] introduces a method which is not altering the input image to a given label but can train the noise from ground up towards a given false label. Resulting in a robust targeted adversarial attack.

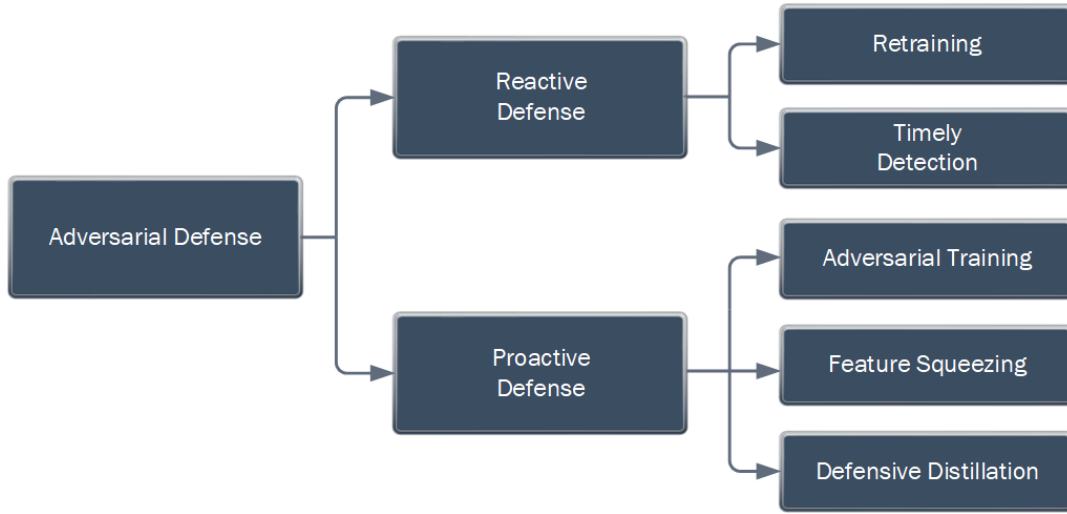


Figure 2.8.: Defense against adversarial attacks after [1]. The main classification of defenses against adversarial attack is determined by the question if the measures can be applied during training or after it.

2.5.5. Jacobian saliency map based attack

In this attack [26] used a targeted saliency map to generate adversarial examples.

2.5.6. Countermeasures against adversarial ML attacks

In order to secure the models against the attacks proposed different methods. Here the main two counter-measurements approaches are detection and prevention. The defense sub-tree contains the sub-features reactive defenses and proactive defenses. In the reactive approach one re-trains or re-configure the ML model after a successful adversarial attack or a detection of such. The main goal here is to protect critical information. The proactive approach implies the inclusion of adversarial examples in the training process. One subcategory of such is called adversarial training. Here one generates the Adversarial examples and trains the ML model with those. This method however can only protect against known examples since adversarial examples do not generalize well by definition. [22]

The other subcategory of proactive defenses is the so called feature squeezing. It was proposed in [27] and describes the process of reducing the feature dimensions available to the adversary. This is done by fusing the samples for different feature vectors.

The third solution is called network distillation. Introduced in [28] the approach hides the gradients before the Softmax function. This way it provides hardening against gradient-based attacks. However this measure was exploited by [24] by realizing an adversarial perturbation technique which evaded this distillation.

2.6. Parallel computing

In parallel computing or parallel processing the computation tasks is distributed and executed on multiple cores or processors. The goal is to utilize more resources to save computation time. The process management task in such scenario generates overhead and has to be minimized. [29]

Traditionally parallel computing was realized by creating sub threads which share the same memory. This could lead to conflicts between processes. A more modern approach which was also encouraged by the rising amount of multi core processors and decreasing memory cost is to use parallel processes for execution. There are two additional groups which categorize multiprocessing or threading. In a synchronous execution the tasks have to be computed in the same order they have started. The main program and other processes are suspended after finishing their part until all sub processes are finished. This is important for computations where the operations are dependent on each other. In asynchronous execution the main process has to be stopped manually. A fixed amount of maximum processes is defined alongside with a queue. As soon as there is one free process slot the next computational part is loaded from the queue and executed. [29]

3. Tools

In this chapter the tools and frameworks which are used are introduced.

3.1. Argus

The raw network capture data is often provided in the form of *.pcap files. These can be processed with Argus [30] to generate connection specific features. The goal is to increase readability by human and more abstract analyzing tools. The framework encapsulates two tools which are used in this work.

3.1.1. Argus command

The first program which comes with the Argus suite is "argus". It processes network capture files or flow data directly from a network interface and provides argus log files which consist of flow status features. The resulting files are the basics for further processing with the Argus suite.

3.1.2. Read Argus command

The *ra* command provides the functionality required to process the argus log files which are generated by the argus command. In the scope of *ra* one can also choose from a variety of features which then will be printed. Table A.1 lists all features which are considered in this thesis.

3.2. NADICS 2.0

NADICS is an open source framework which was developed at the TUM and is a machine learning engine based on the scikit-learn. Basically NADICS works as a tool to increase usability by providing an CLI tool set to work with network capture data in form of Comma Separated Values (CSV) files. It can be used to classify attacks on given datasets. The system requires python 2.7 to run. The modularity of the framework enables easy implementation of new models and core functions. The main target user group are researchers who want to use the system as a standardized approach to evaluate the performance of ML algorithms against given datasets. NADICS is able to perform clustering and classification tasks. However in my work I focus only on the classification part.

The basic structure of the system consists of a preprocessing and a Training unit which are used to create a model. Figure 3.1 shows the pipeline for a simple training scenario.

Files	Description
Training/Test datasets	*.csv files which contain the training and test data. Those are already separated to ensure that no training operations are performed on the test dataset.
layout file	*.ini style config file to handle the layout interpretation of the training and testing files.
config file	*.ini style config file to define path configuration of other project specific files and training parameters like random state.
preprocessed files	Files which are generated after a dataset is preprocessed for the training. saved time later.
model file	Trained models can be saved in model files for later retrieval. The Files are in Python pickle format.
results file	Results of the training process can be stored in separate files for later retrieval.

Table 3.1.: File sets which has to be provided to or are generated by NADICS

3.2.1. Project structure

Originally every dataset in NADICS consisted of at least three file sets located at different directories. Table 3.1 summarizes all for our project relevant files in addition to figure 3.1.

3.2.2. Preprocessing

The directive in the preprocessing block is to parse the user input and the provided files. After preprocessing the files the user can choose the result should be stored as a pickle dump for later retrieval. Furthermore there is a visualisation module which is capable of plotting feature importances for the random forest classifier. The following classification ML algorithms from the scikit-learn are available to choose from: AdaBoostClassifier, DecisionTreeClassifier, RandomForestClassifier, KNeighborsClassifier, GaussianProcessClassifier, GaussianNB, GradientBoostingClassifier, BernoulliNB, MultinomialNB, MLPClassifier, SGDClassifier.

3.2.3. Training

This module takes the training and testing datasets together with the model and performs the methods require for training testing and evaluation.

3.3. scikit-learn

Scikit-learn is a framework which offers common data science algorithms in a standardized manner. It comes also with various datasets and with methods to bring new datasets into

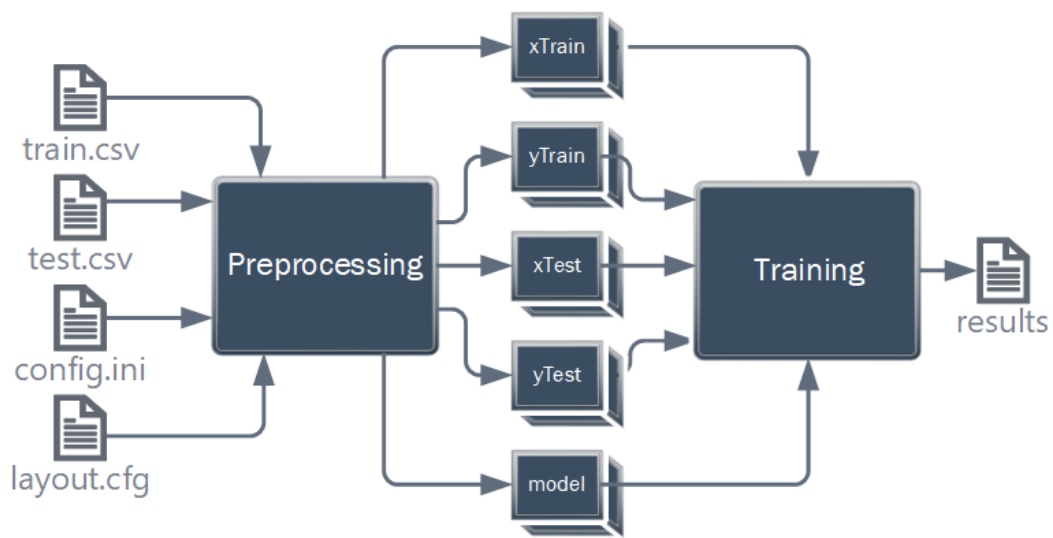


Figure 3.1.: The basic pipeline of NADICS. The input files are provided by the user and processed to `xTrain`, `xTest`, `yTrain` and `yTest` pandas data frames. The dataframes with `x` as prefix refer to the selected features which are used in the training and testing process. The dataframes with the prefix `y` provide the selected label correspondingly. After the training process the results can be stored in a textfile in the form of a line which provides information about accuracy and f1 score

the desired format. An estimator, the core function for a ML algorithm in the framework, offers two functions. One is $fit(X, y)$ to train the estimator and $predict(X)$ is used to make predictions. The trained model can be saved by dumping it into a pickle file. [17]

Additionally the kit offers common performance measurements like the ones introduced in 2.3. Automl techniques like grid search and various normalization techniques are also supported natively. [17]

3.4. Pytorch

This ML framework sets it's target in maximization of both usability and speed of ANN ML development. It enables model and process definition in Python and execution on either CPU or GPU. A variety of predefined and trained models is also provided and can be used right away [31]

4. Dataset preprocessing

The basis of the research are the datasets. For the sake of variety two network datasets were derived which represent the communication in two different network environments. The first set is CICAndMAL2017 [32] which represents the communication of Android devices with the WAN. It communicates on all levels of the OSI model. The second dataset is the Secure Water Treatment (SWaT) dataset which represents the communication between control devices in a water treatment unit. The Supervisory Control And Data Acquisition (SCADA) network which is used by the system consists of multiple layers. The dataset contains data from the lowest layer 0 in which the protocol EtherNet/IP is used [33]. Further information about the structure can be found in chapter 4.2. The following chapter 4 describes how the dataset data is preprocessed before the utilization in NADICS.

4.1. CICAndMal2017

In order to provide the most recent application scenario we use the dataset from [32]. The samples in this dataset were collected from physical smart phone devices. The behaviour on the phones was generated by scripts which imitated normal phone usage like phone calls and utilizing Short Message Service (SMS) alongside with Global Positioning System (GPS) spoofing and web browsing. Every phone had also a Gmail, Facebook, Skype and Whatsapp account. The normal behaviour of the phones was recorded to *.pcap files which act as the entry point of our work. After that every phone was infected with a malware from the malware pool provided with the dataset in the form of *.apk files. The resulting network communication was recorded. The directory structure of the pcap sub dataset is illustrated in table 4.1.

Since the data only consists of *.pcap files an implementation of algorithms was necessary to extract features and provide labeled data in form of *.csv files. Those output files can be directly used in NADICS. Figure 4.1 illustrates the architecture of the two scripts *pcaps2argus_csvs.sh* and *argus_csvs2train_csvs.py* which are designed and implemented to preprocess the data. To extract the training features Argus which is introduced in chapter 3.1 was utilized. The file handling through it is contained in the script *pcaps2argus_csvs.sh*. Argus takes *.pcap files as input from the *1_pcaps* directory and provides log files into *3_argus_logs*. Those are then processed with the *read argus (ra)* command which outputs *.csv files. In this step one can decide which features have to be written in the output file. Table A.1 shows the features we considered for our approach. The directory *2_processed_pcaps* was used to store pcaps after merging or splitting, which was also considered during this work.

The resulting csvs are processed further by the *argus_csvs2train_csvs.py* script which applies

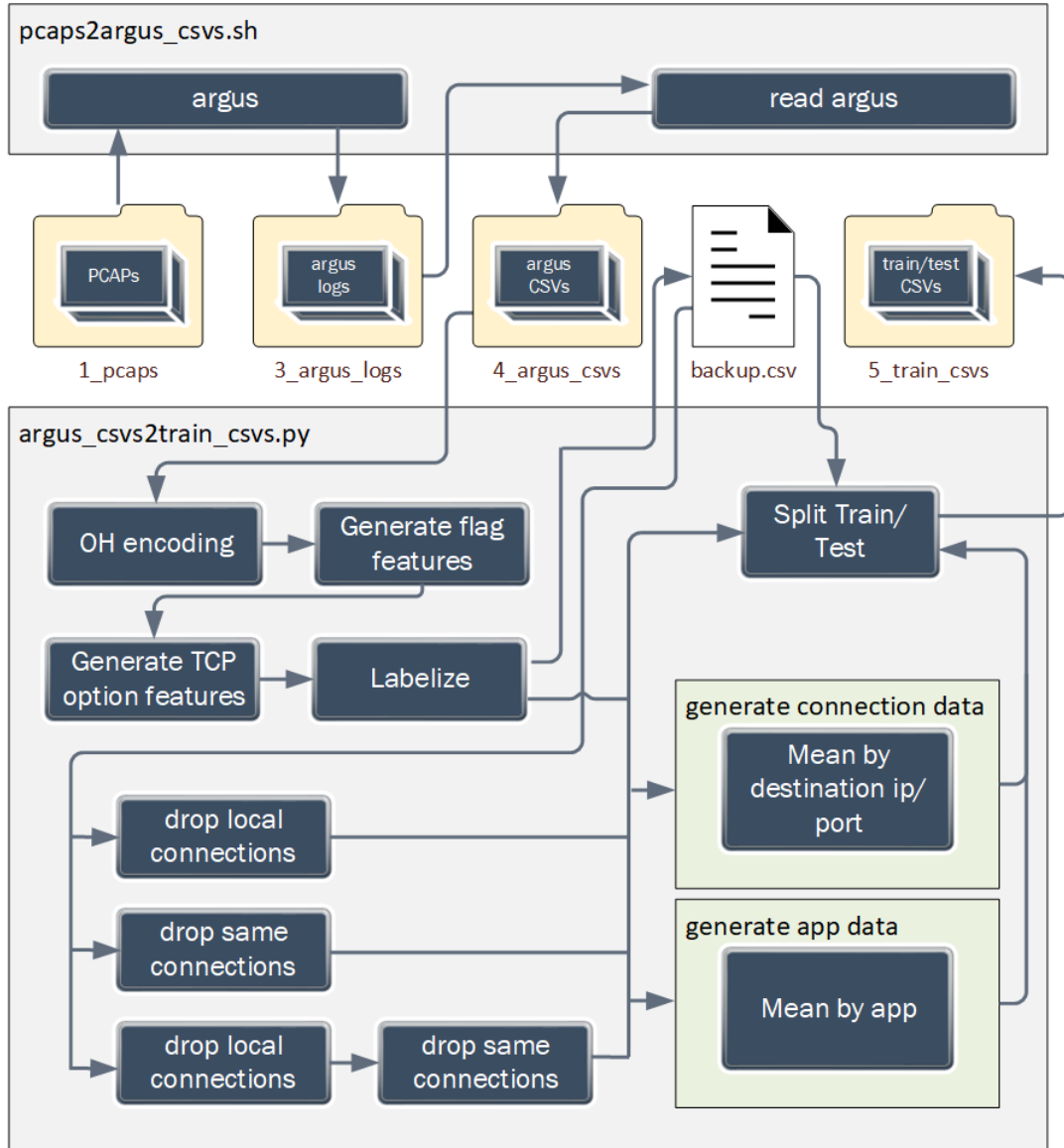


Figure 4.1.: The PCAP processing pipeline for the CICandMAL2017 dataset.

4. Dataset preprocessing

Category	Adware-PCAPs	Benign-PCAPs	Ransomware-PCAPs	Scareware-PCAPs	SMSmalware-PCAPs
Family	Dowgin	2015	Charger	AndroidDefender	Beanbot
	Ewind	2016	Jisut	AndroidSpy.277	Biige
	Feiwo	2017	Koler	AVforandroid	fakeinst
	Gooligan		LockerPin	AVpass	fakemart
	Kemoge		Pletor	FakeApp	fakenotify
	koodous		PornDroid	FakeApp.AL	jifake
	Mobidash		RansomBO	FakeAV	mazarbot
	Selfmite		Simplocker	FakeJobOffer	Nandrobox
	Shuanet		Svpeng	FakeTaoBao	plankton
	Youmi		WannaLocker	Penetho	smssniffer
				VirusShield	zzone

Table 4.1.: PCAP directory structure of the CICandMal2017 dataset

One-Hot encoding (OH encoding) onto the columns *Dir* and *Proto*, then generates features from the *Flgs* and *tcpopt* columns in the argus CSVs. Since the *Flgs* and *tcpopt* columns can be divided in distinct characters which indicate different flags or states which are seen during connection they were splitted and divided in separate columns, each for every indicator. For the labeling three systems were implemented. Those are binary, category and family. The binary category has the labels benign and malicious, where all data from the Benign-PCAPs subdirectory is labeled with "benign" and everything else with "malicious". The category and family labels mirror the corresponding directory structures with one exception. The Benign-PCAPs directory contains year sub directories. For the rest of this work we consider them as a whole and define the benign family which contains all benign apps. Every other family item is considered as a value in the family label. The distinct apps with the corresponding labels can be derived from table 4.1. The resulting argus csv contains 3022614 tuples and is stored in the *backup.csv* file.

The next step was to provide different configurations of refinement onto the data to counteract similar connections in the malicious and non malicious activities and provide better labels. The effects of those steps are measured and discussed in chapter 6. Those approaches were applied in all possible combinations on the data from the previous step to generate a measurable difference. The following two approaches were implemented in this scope:

drop local connections The assumption is that the malware does not interact with the local network and thus connections like DHCP or ARP request can be dropped. It is possible that there is still malicious traffic in the local network subset. For example a ransomware which is infecting devices in the same network.

drop same connections In the original datasets multiple host/port combinations are present in both malicious and benign data tuples. The assumption is that during the creation of datasets background activities were performed by the phones. This could include but is not limited to Domain Name System (DNS) requests, update requests or Network Time Protocol (NTP) synchronizations.

To build the final datasets we introduced the following approaches:

flow wise In this approach the flow features from the previous step were provided to the next step without altering them. The tuples are the direct output of Argus.

connection wise Argus data was used to extract connection specific features. To provide a data tuple per every TCP connection all possible combinations of destination IP/port were grouped and meant for every app. The generated meant tuples represented one combination each.

app wise In this scenario the Argus output for every app was meant leaving one value for each feature in the form of an tuple per app. This method reduces the dataset drastically to 2126 samples.

After splitting the data into training and test datasets basic feature selection was performed on the resulting *.csv files. This includes removing all features with zero variance, all sources of OH encoding and other processing which renders the original column useless, address and port information and every feature of the one hot encoding group which occurs less than 10 times in the whole dataset. Table 4.2 summarizes the changes. Furthermore all NaN values were substituted with zeros. Those occurred when there was no OH encoding occurrence in one source *.csv file for a specific encoded value.

4.2. Secure Water Treatment (SWaT) Dataset

A SWaT unit is an operational scaled down version of a water treatment plant. The purpose of the replication is to provide a testing environment for development of the big plants. The dataset we utilize is generated from the wired network communication inside the plant. The network consists of PLCs, a SCADA system and Human Machine Interfaces (HMIs). The network communication on level 0 was recorded during an attack on one or more network components and under normal circumstances and labeled correspondingly. [34] [35]

The whole testing period spans to 11 days. The system was running the whole time. Since the plant requires seven days to reach its normal state all attacks were performed in the last four days. The data was recorded every second. This dataset is not focused on network communication itself but the payload, which is the values of sensors and actuators. A detailed list with descriptions for every feature can be found in A.3 and [34].

There are multiple attacks types which were performed during the attack phase. In [34] those are categorized based by the amount of attacked points in a Cyber Physical System (CPS) and the amount of stages which are attacked simultaneously. The most attacks were single point attacks. After each attack the system was given time to return to its normal state. [34]

Since the data is provided in the form of one *.xlsx file the preprocessing consisted in converting the file to *.csv, dropping zero variance features and split the resulting table into a training and testing set. The removed features with zero variance are: *P202*, *P301*, *P401*, *P404*, *P502*, *P601* and *P603*.

Variable(s)	Reason
SrcAddr, DstAddr, Sport, Dport	Host/Protocol specific values. Not good for generalizing
Seq, Rank	Incrementing numbers. No value for our purposes
Trans	Variance = 0
Flgs	Are handled by the corresponding OH encoding
StdDev	Variance = 0
Proto	Handled by the corresponding OH encoding
TotAppByte, SAppBytes, DAppBytes	Variance = 0
PCRatio	Variance = 0
Retrans, SrcRetra, DstRetra	Variance = 0
pRetran	Variance = 0
Flgs_0_none, Flgs_5_none, Flgs_6_none, Flgs_7_none, Flgs_8_none	Variance = 0
Flgs_1_*	Exclusive or with Flgs_1_e
Proto_rtp	One occurrence in the whole dataset
Flgs_2_T	4 occurrences in the dataset
Flgs_2_None	considering the sparse occurrence of Flgs_2_T it is antivalence to Flgs_2_U
Flgs_3_&	one occurrence in the dataset
Proto_udt	2 occurrences in the whole dataset where it is != 0
Flgs_4_none, Flgs_4_D	7 occurrences of Flgs_4_D in the whole dataset considering the sparse occurrence of Flgs_3_D it is inverse to Flgs_4_S

Table 4.2.: Variables which were removed from the CICancVAL2017 dataset before feature selection

5. NADICS 3 development

One of the core challenges in this work is the development of the next major version of NADICS. One of the major changes was the upgrade to python 3.8 since python 2 reached its end of life on 01.01.2020. Furthermore the tool was extended by the adversarial examples mode. The development history of the core changes is summarized in the sections. Further development was made to implement Normalization methods, One-Hot-Encoding.

5.1. Upgrade to Python 3

In order to ensure future compatibility and usability NADICS was upgraded to the latest Python version. Luckily most of the code was already in python compatible style. The most significant incompatibilities were the ones with *print* commands and library structure changes.

5.2. Project structure

In order to increase modularity and project overview, all of the files from table 3.1 were contained in a single directory and moved to the *data* sub-directory in NADICS. This included not only the configuration file adjustments but also some adjustments in the code since some files could only be stored in other directories despite the configurations in the project configuration file.

5.3. Training tables header

Originally only *.csv files without a header could be processed by NADICS this was most probably the reason why one had to provide the column names in the layout file with the keyword *fullHeader*. A header is the first line in the file containing the names of the columns instead of the values. This makes the system less intuitive and more sensitive to human error since switched column names could lead to wrong interpretation from the system. The previous functionality was preserved but also the binary keyword *csvHeader* was added. If set to true the system will interpret the header inside the *.csv files and select the columns provided with the keyword *fullHeader* as a subset from the original data.

5.4. Label encoding and handling

With increasing quantity of labels the initial labeling system provided by NADICS becomes less intuitive. The labels had to be numerical because of the handling mechanisms of scikit-learn. For this reason we utilized the LabelEncoder provided by the scikit-learn framework to encode the labels provided by the read in *.csv files. This extends the possible values for labels to the alphanumeric value space. However the resulting encoder has to be utilized at any point where the labels are needed to get the original ones. This includes but is not limited to statistics like printed out accuracies per class or plots.

The other important change was the handling of the label column itself. In the previous NADICS system the user was required to provide the amount of classes for a classification task. This is rather intuitive because one usually does not choose a subset of provided classes leaving the other classes out. For this reason the project layout file parsing was adapted in such a way that one could provide a label column with the *labelHeader* keyword. The label is then encoded automatically based of the amount of distinct values in this columns.

5.5. Data imbalance handling

The imbalance handling mechanism downsampling introduced in chapter 2.4.2 was implemented. The functionality can be enabled with the binary setting *equalize* in the project layout file. If set all classes will be brought at the same quantity as the label with the fewest occurrences. This is handled in the *preprocessor.py* script.

5.6. Normalization

To prevent feature scaling issues from happening. There are different approaches which differ in terms of outlier handling. The tree approaches which were considered for this work were the StandardScaler, MinMaxScaler and RobustScaler which are all pre-implemented in the scikit-learn framework. The RobustScaler was found to provide the best performance so it was implemented into the pre-existing *normalizer.py* script. The normalization is fitted on the training dataset and transformed on the test set to prevent dependence on the test set.

5.7. Principal Component Analysis (PCA)

PCA which was introduced in chapter 2.2.9 was implemented into *normalizer.py* allowing to perform it on the dataset. The implementation from sklearn kit was used. The fit function is performed on the training and the transform function on the test dataset. The functionality can be enabled with the *-P* flag from the command line.

5.8. Feature Selection

To tackle the curse of dimensionality Feature Selection (FS) methods based on covariance measurements were introduced. Concretely the `corr()` method from the Pandas framework was used to generate a Pearson correlation matrix over all features and the label. The absolute values from this matrix represent the similarity between distinct features and label. The filter based FS method consists in dropping features which have a great similarity to other features and the ones which have a small similarity to the label. The threshold for dropping out features is defined as the hyper-parameters for this FS method. The matrix visualization was implemented in the `visualize.py` script. Furthermore recommendations are printed in the shell now if a low similarity with the label is identified or the variance of the feature is zero. The corresponding functionality was implemented in the `classify.py` script.

5.9. compare module

To compare the trained models with each other a corresponding visualization module was added in the form of the `compare.py` script. The script takes result files as input and plots bar charts which sets the values f1 score, recall and precision in relation. For this module the results saving routine was also altered. It can feature multiple models at once now and is saved in the form of a pickle dump of a directory in which the metrics are stored.

5.10. Model adapter to PyTorch

Besides of the Multi Level Perceptron model provided by scikit-learn framework, there are no out of the box ANN implementations in NADICS. To continue using NADICS an adapter which features the basic model functions described in chapter 3.3 was implemented. The implementation is located in the `pytorch.py` file which is located in the main NADICS directory. On model creation various parameters can be set. The model can be trained then by providing the features and labels as separate numpy arrays to the `fit(X, y)` function. The `predict(X)` function is used to classify new data.

5.10.1. Deep Neural Network

The network itself is defined by the class `TorchNetwork`. In this work only fully connected DNNs are considered. The constructor can be called with dimensionality parameters input/output dimensions and hidden layers. This ensures flexibility in definition and enables the possibility for grid search over different network architectures, with different amounts of hidden layers and neurons per layer. The new classifier can be called with the `DNN` argument in the CLI.

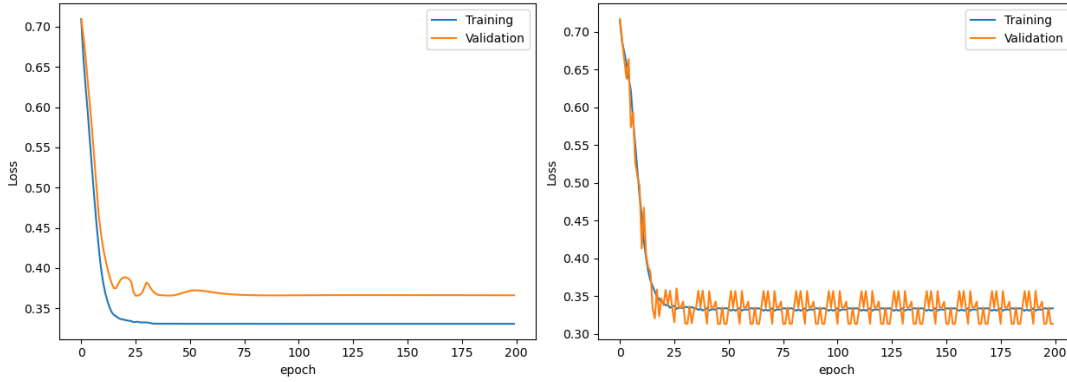


Figure 5.1.: Different behaviour during training dependent on whether basic CV is used (Right) or not (Left)

The TorchNetwork class is accessible through the TorchClassifier class. It encapsulates the network providing necessary functions to train the network and perform inference in a standard scikit-learn way. The TorchClassifier class features the two standard scikit-learn functions $fit(X, y)$ and $predict(X)$ to ensure compatibility. The constructor can be called with the parameters learning rate, momentum, max epoch and debug to ensure conductivity at definition. The dimensionality for the in and output layers is calculated automatically after the features and label lists are provided to the $fit(X, y)$ function. Furthermore the function $predict_{percentages}(X)$ was introduced. It predicts probabilities per class rather than returning a class like the $fit(X, y)$ function.

after the $fit(X, y)$ function is called by nadics the data is prepared and DNN is initialized. This consists of moving the numpy arrays to torch tensors and the initialization of the network with the corresponding shapes which are derived from X and y . The Adam optimizer which is also provided by PyTorch is initialized with the parameters provided to the constructor. During implementation multiple functionalities of the Cross-validation (CV) were explored by trying it on the CICandMAL2017 app dataset. The behaviour of the algorithm can be controlled by providing the k parameter to the $fit(X, y)$ function. First an implementation were made where the fold for the validation dataset was switched every epoch. The resulting behaviour can be observed in figure 5.1. The loss for the validation set oscillates. In the left image a fixed validation set of 100 samples was used.

Since the previous implementation can't be used in a meaningful way, the variant where every fold can be used only one time during training was implemented. The k -fold CV. The results can be observed in figure 5.2. However in this form the validation curve was not found to be usable. It encourages a batch like learning behaviour which could have performance advantages. But those were not explored further.

Finally a logging system was introduced to the script which logs the learning performance of the DNN and can present it in the form of a learning curve.

To ensure deterministic behaviour of the DNN the seed parameter was set to seven. This is

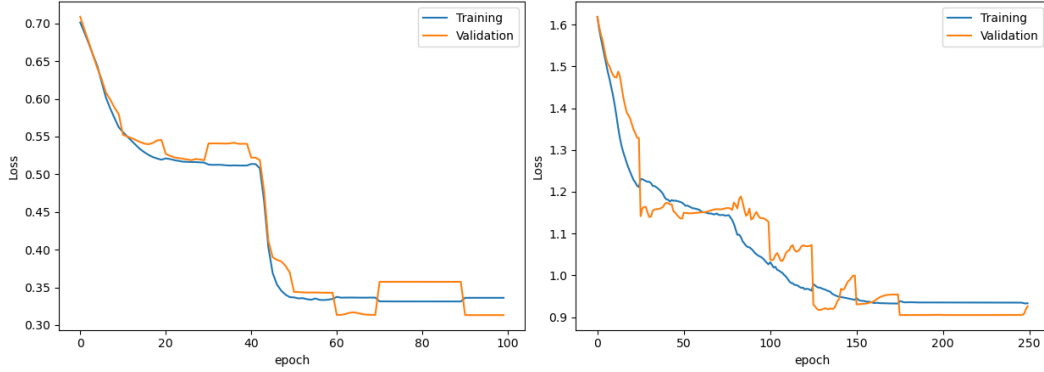


Figure 5.2.: Learning curves for the models which were trained with the k-fold CV. Binary classification (left) categorical (right)

required to generate reproducible results which are necessary for comparison of the trained networks. Every function in the PyTorch framework uses this the seed to generate pseudorandom numbers. Normally it is set randomly at the beginning. By setting it to a fixed value the training process becomes deterministic.

5.10.2. FGSM and BIM attacks

The FGSM attack is implemented in the function $fgsm(X, y, epsilon)$ which is contained in the TorchClassifier class. The code is mostly derived from the FGSM example from [36]. Additionally to the training data the function requires the epsilon parameter defined in the formula 2.5.1. The function performs the FGSM attack on the provided dataset and returns any successful tuples. A successful tuple is in this case any tuple which can be manipulated by the function in such a way that the predicted label is not correct. As a prerequisite to the attack the function ensures that the label is predicted correctly in the first place.

The BIM attack is also located in in the TorchClassifier class and is realized by calling the FGSM function a definable number of times. The user can define epsilon values and the amount of Basic Iterative Method (BIM) runs inside the function to perform a grid search over the parameters. the results are stored in a Python directory and can be visualized by the function `show_adversarial_result` which is located in the `visualize.py` file. The visualization plots the results of the grid search as a Seaborn line chart.

5.10.3. Counter-measurements

As the counter measurement for the introduced attack the adversarial training was implemented. The procedure consists of storing the preprocessed data and train the classifier on it. The classifier is evaluated on the testing dataset. After the training the BIM attack

is performed on the training and testing datasets and the adversarial examples from the training set for a specific epsilon value and BIM run are appended to the training table. A new classifier is trained on the resulting dataset and again evaluated against the adversarial testing dataset. The increased robustness can be measured by comparing accuracies.

6. ML Experiments

All adversarial experiments were performed in the scope of this chapter. The chapter is separated in three parts. First the overall methodology of the experiments is defined alongside with the parameters. The experiments are executed on each dataset in the corresponding other chapters.

6.1. NADICS configuration

Initially FS methods introduced in chapter 2.2.8 were executed on the datasets. All features that express a similarity higher than 0.95 to other features were excluded from the datasets leaving only one representative feature to train on for each group. Additionally all features which have a lower similarity than 0.01 with the label were excluded from training. The experiments were conducted without the PCA functionality explained which was implemented earlier. The downsampling functionality was used to provide equal class distributions.

The ML learning algorithms Random Forest, ADA boost, Gradient Boost, k-Nearest Neighbors and Support Vector Machines were used as a performance metrics for the datasets them self. The goal is to determine reachable accuracy as a contrast to [32] for the CICandMal2017 dataset. The procedure was also executed on SWaT. For both datasets similar hyperparameters were used in the scope of classical ML algorithms. The random state was set to 10 to all algorithms to ensure determinism. For the SVM model the SVC version with the linear kernel was used.

The next step is to train the DNN. The same network structure was used for all experiments. The input and output layers were adapted automatically to the features and label configurations. The hidden layer group consisted of two layers with the sizes 60 and 30. Learning rate was set to 0.01. All training steps were performed without a validation dataset since the data for the CICandMAL2017 app dataset was too sparse. Adam was used as the optimizer. This makes the number of epochs the only hyper-parameter for the DNN model.

Thread model for the attacks is defined as follows: **Adversary knowledge:** Evasion, white box, non-targeted attacks are considered **Adversary Goal:** The adversaries target is to hide the attack traffic from the NIDS. Namely in the classification scenario the attacker tries to develop an adversarial traffic which is classified as BENIGN instead of MAIL or the other way around. To perform the adversarial, the BIM method is used. For the special case of one run it is equivalent to the FGSM method. For this reason both attacks are covered in the experiments. The performance of the attack is evaluated for every BIM run and every epsilon configuration from the hyperparameter space. The following epsilon values were considered (0.01, 0.05, 0.1, 0.15, 0.2, 0.25, 0.5, 0.75, 1). The results are presented in the form of figures which

Removed	Similarity with	Reason
Dur, RunTime, Sum, Min, Max	Mean	1
TotPkts, DstPkts, DstBytes	TotBytes	> 0.95
DstLoss	Loss	0.964625
Proto_arp	Dir_who of	1
tcpopt_M, tcpopt_s	tcpopt_w	1
SynAck	TcpRtt	0.990371
TotBytes	DstBytes	0.997179
Flgs_3_s	pLoss	0.993267
Flgs_2_U	Proto_icmp	0.992084
sTtl	label	0.007811
SrcBytes	label	0.006431
Dir_who, Proto_llc	-	Used for local communication

Table 6.1.: FS based drop out of features for the CICandMAL2017 dataset

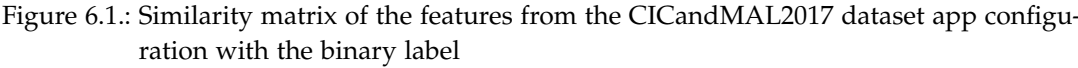
set the number of BIM iteration in relation to the adversarial examples found for the different values for epsilon. After the adversarial attack the adversarial training process was performed. For each retraining experiment the epsilon value for the adversarial examples was defined. The adversarial examples were stored for both training and test datasets. To prevent the influence of the test dataset only the training dataset examples were used for adversarial training. Both adversarial trained and original models were evaluated against the adversarial and original test datasets. Finally the same adversarial attack is performed on the adversarial trained models to explore their sensitivity against further adversarial attacks.

6.2. CICandMAL2017

For the CICandMAL2017 dataset the 12 possible datasets which we generated during preprocessing were tested. Those are the combination of flow, connection and app with no drops, dropped local traffic and dropped connections which occur in both malicious and benign binary label groups.

6.2.1. NADICS Preprocessing

The preprocessing was performed on the app dataset with no drops since it featured the highest similarities of features to binary label. Figure 6.1 shows the correlation matrix of the features and label. Table 6.1 shows the features which were dropped with the corresponding reason to do so. After the preprocessing 38 features are left. Figure 6.2 illustrates the similarity of distinct labels to the features in a multi label scenario of the app dataset with no drops.



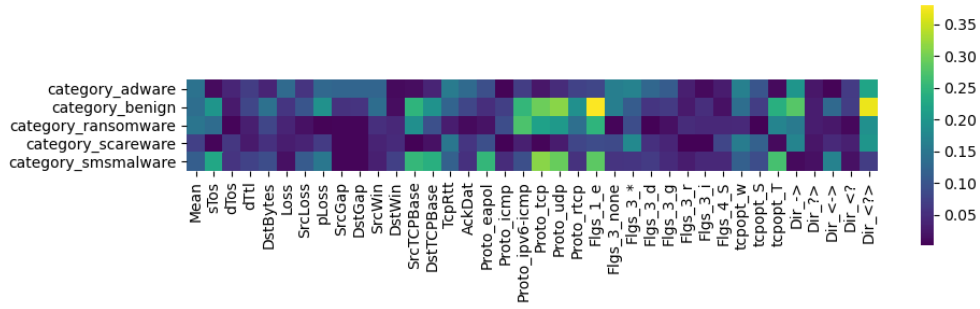


Figure 6.2.: Similarity of distinct labels to preselected features

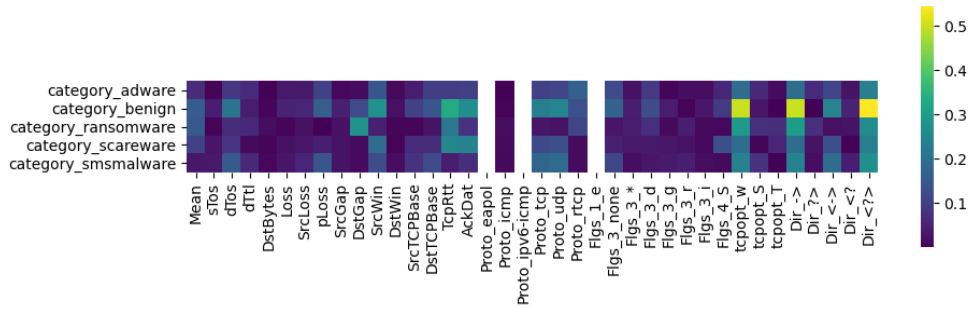


Figure 6.3.: Similarity of distinct labels to preselected features with dropped same connections

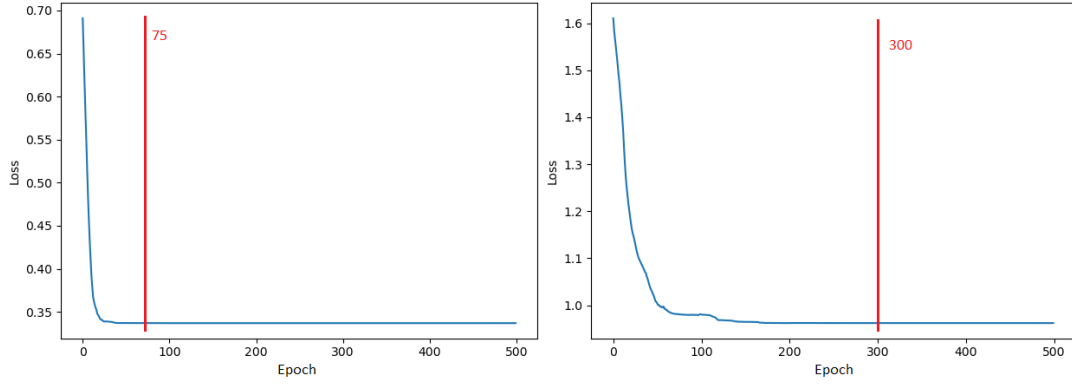


Figure 6.4.: Epoch selection for DNN training for the CICandMAL2017 dataset labels binary (left) and category (right)

6.2.2. Classical ML

Table 6.2 contains the performance metrics which were used for the dataset selection. The dataset which performs in both binary and categorical labeling had to be determined. To achieve this a score system was defined which consists of the sum of all f1 scores per classical ML algorithm leaving out ADA boost because it did not performed well overall in the categorical scenario. This makes the App dataset with dropped local traffic the choice for further training.

Interestingly all datasets performed much worse with dropped same connections which indicates that the behaviour towards public services is important too in the scope of malware classification. Figure 6.2 can be compared to the analogue figure 6.3 which was generated from the app dataset with dropped same connections. One can observe that the features "Proto_eapol" and "flgs_1_e" are among others completely missing in the second case. Since those features had a relatively high similarity to the labels this is the most probable reason for the poor performance on the dataset with dropped same connections. However the research in this direction is outside of the scope of this thesis.

6.2.3. Deep Neuronal Network

Considering the small amount of data 20% was used as the test set. The other 80% was used for training. The possibility of utilizing k-cross validation with 10 folds to counteract the data sparsity was explored, but dropped due to data sparsity. To determine the optimal epoch to stop training on both models were trained to the 500 epoch. The value was chosen based on the loss development one can observe in figure 6.4. From it the values 75 for binary and 300 for category labels were derived. The final models had an accuracy of 97% for binary and 85% classification problem.

6. ML Experiments

Binary		Flow					Connection					App				
		RF	ADA	GB	kNN	SVM	RF	ADA	GB	kNN	SVM	RF	ADA	GB	kNN	SVM
Accuracy	no Drops	0.70	0.57	0.60	0.61	0.54	0.68	0.59	0.62	0.59	0.56	0.99	0.99	0.99	0.90	0.92
	local	0.73	0.58	0.61	0.62	0.55	0.68	0.59	0.62	0.60	0.57	1.00	1.00	1.00	0.85	0.93
	same	0.73	0.58	0.61	0.62	0.55	0.76	0.72	0.74	0.70	0.70	0.95	0.93	0.96	0.82	0.88
	both	0.91	0.80	0.83	0.82	0.71	0.78	0.76	0.77	0.74	0.74	0.96	0.94	0.96	0.85	0.86
Precision	no Drops	0.70	0.57	0.60	0.61	0.54	0.68	0.59	0.62	0.59	0.56	0.99	0.99	0.99	0.91	0.92
	local	0.73	0.58	0.61	0.62	0.55	0.68	0.59	0.62	0.60	0.57	1.00	1.00	1.00	0.88	0.93
	similar	0.73	0.58	0.61	0.62	0.55	0.76	0.72	0.74	0.70	0.71	0.95	0.93	0.96	0.83	0.88
	both	0.91	0.80	0.83	0.82	0.72	0.78	0.76	0.77	0.74	0.74	0.96	0.94	0.96	0.85	0.87
f1 score	no Drops	0.70	0.57	0.59	0.61	0.54	0.68	0.59	0.62	0.59	0.56	0.99	0.99	0.99	0.90	0.92
	local	0.73	0.58	0.61	0.62	0.54	0.68	0.59	0.62	0.60	0.57	1.00	1.00	1.00	0.85	0.93
	same	0.73	0.58	0.61	0.62	0.55	0.76	0.72	0.74	0.70	0.70	0.95	0.93	0.96	0.82	0.88
	both	0.91	0.80	0.83	0.82	0.71	0.78	0.76	0.77	0.74	0.73	0.96	0.94	0.96	0.85	0.86
Recall	no Drops	0.70	0.57	0.60	0.61	0.54	0.68	0.59	0.62	0.59	0.56	0.99	0.99	0.99	0.90	0.92
	local	0.73	0.58	0.61	0.62	0.55	0.68	0.59	0.62	0.60	0.57	1.00	1.00	1.00	0.85	0.93
	similar	0.73	0.58	0.61	0.62	0.55	0.76	0.72	0.74	0.70	0.70	0.95	0.93	0.96	0.82	0.88
	both	0.91	0.80	0.83	0.82	0.71	0.78	0.76	0.77	0.74	0.74	0.96	0.94	0.96	0.85	0.86
Category		Flow					Connection					App				
		RF	ADA	GB	kNN	SVM	RF	ADA	GB	kNN	SVM	RF	ADA	GB	kNN	SVM
Accuracy	no Drops	0.50	0.28	0.33	0.35	0.24	0.42	0.28	0.33	0.28	0.24	0.79	0.39	0.80	0.60	0.70
	local	0.55	0.30	0.35	0.37	0.26	0.43	0.29	0.34	0.29	0.24	0.79	0.22	0.82	0.59	0.70
	same	0.82	0.57	0.68	0.66	0.44	0.51	0.40	0.48	0.39	0.37	0.73	0.54	0.73	0.57	0.62
	both	0.82	0.56	0.69	0.67	0.44	0.52	0.42	0.48	0.41	0.36	0.73	0.40	0.76	0.61	0.55
Precision	no Drops	0.49	0.28	0.34	0.35	0.24	0.41	0.27	0.33	0.28	0.24	0.79	0.41	0.80	0.62	0.70
	local	0.55	0.29	0.36	0.37	0.26	0.43	0.28	0.33	0.29	0.24	0.78	0.22	0.83	0.63	0.71
	same	0.83	0.58	0.69	0.67	0.49	0.51	0.40	0.48	0.39	0.37	0.74	0.54	0.73	0.57	0.60
	both	0.83	0.57	0.69	0.67	0.48	0.53	0.42	0.49	0.41	0.38	0.73	0.43	0.76	0.64	0.55
f1 score	no Drops	0.49	0.27	0.33	0.35	0.24	0.42	0.26	0.32	0.28	0.21	0.79	0.37	0.80	0.59	0.69
	local	0.55	0.29	0.35	0.37	0.24	0.43	0.27	0.33	0.29	0.23	0.78	0.13	0.82	0.58	0.70
	same	0.82	0.57	0.68	0.66	0.44	0.51	0.39	0.47	0.38	0.36	0.73	0.54	0.72	0.56	0.60
	both	0.82	0.56	0.69	0.67	0.44	0.52	0.39	0.47	0.40	0.33	0.73	0.40	0.76	0.61	0.54
Recall	no Drops	0.50	0.28	0.33	0.35	0.24	0.42	0.27	0.33	0.28	0.24	0.79	0.39	0.80	0.60	0.70
	local	0.55	0.29	0.35	0.37	0.26	0.43	0.29	0.34	0.29	0.24	0.79	0.23	0.82	0.59	0.70
	same	0.82	0.57	0.68	0.66	0.44	0.51	0.40	0.48	0.39	0.37	0.73	0.54	0.73	0.57	0.62
	both	0.82	0.56	0.69	0.67	0.44	0.52	0.42	0.48	0.41	0.36	0.73	0.40	0.76	0.61	0.55

Table 6.2.: Classical ML learning results for the CICandMAL2017 dataset

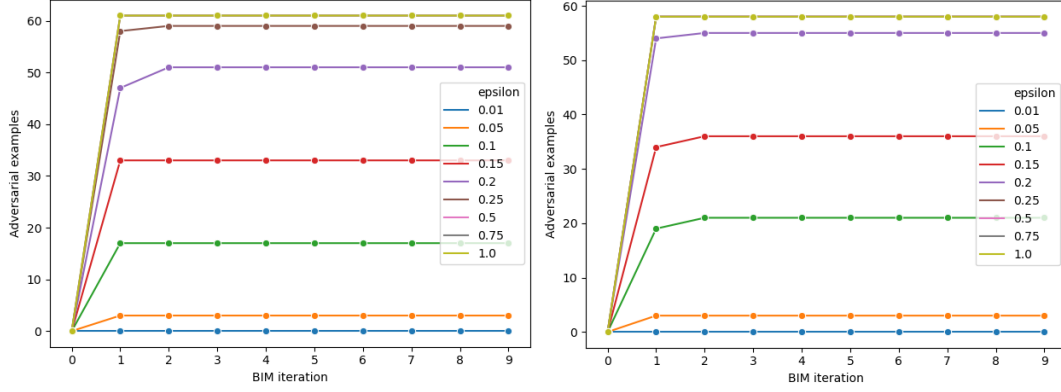


Figure 6.5.: Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the binary classification problem of the CICandMAL2017 dataset

Adversarial attack and training

For the adversarial training the epsilon values 0.05 and 0.15 were chosen for the binary and category model. Figures 6.5 and 6.6 illustrate the final results. One can observe that the sensitivity to further adversarial attacks is increasing for the adversarially trained models in both cases. The fact that the lines align on the top of the charts for higher epsilon values, suggest that there is a boundary which defines the maximum amount of adversarial examples can be generated for a given dataset.

6.3. Secure Water Treatment (SWaT) Dataset

6.3.1. NADICS Preprocessing

Figure 6.7 illustrates the similarity between the features and the binary label. The features with high similarity to other features and low similarity to label were removed from the feature space. Table 6.3 summarizes the process.

6.3.2. Classical ML

The CICandMAL dataset was tested on classical ML algorithms in order to explore and determine the best accuracies. Table 6.4 summarizes the results.

6.3.3. Deep Neuronal Network

Figure 6.8 shows the resulting learning curve. For the training 300 epochs were used and reached an accuracy of 92% in the only available binary classification problem.

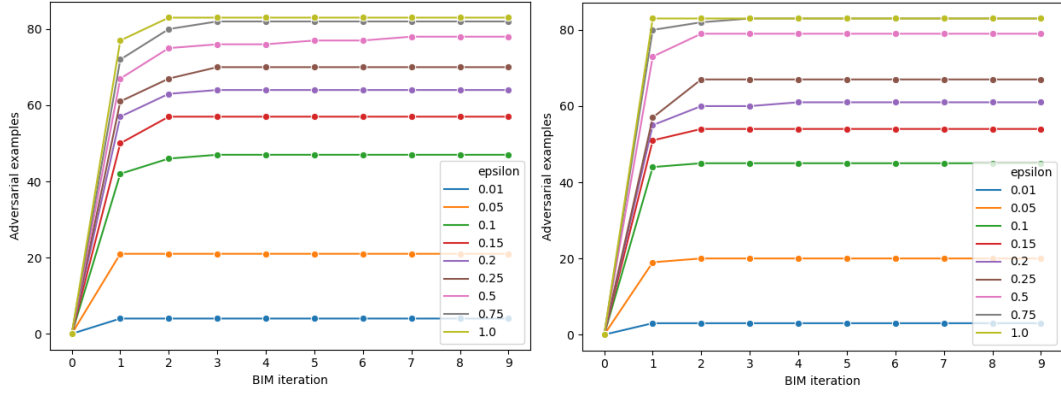


Figure 6.6.: Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the category classification problem of the CICand-MAL2017 dataset

Removed	Similarity with	Reason
P403	label	0.006
P205, MV201, FIT201, P101	P203	> 0.95
MV101	FIT101	> 0.95
P206	P204	> 0.95
P302	FIT301	> 0.95
FIT501, FIT502, FIT503, P501, PIT501, PIT503, AIT402, P402, UV401, FIT504	FIT401	> 0.95

Table 6.3.: FS based drop out of features for the SWaT dataset

	RF	ADA	GB	KNN	SVM
Accuracy	1.00	0.94	0.99	0.99	0.89
Precision	1.00	0.94	0.99	0.99	0.90
f1 score	1.00	0.94	0.99	0.99	0.88
recall	1.00	0.94	0.99	0.99	0.88

Table 6.4.: Classical ML learning results for the SWaT dataset

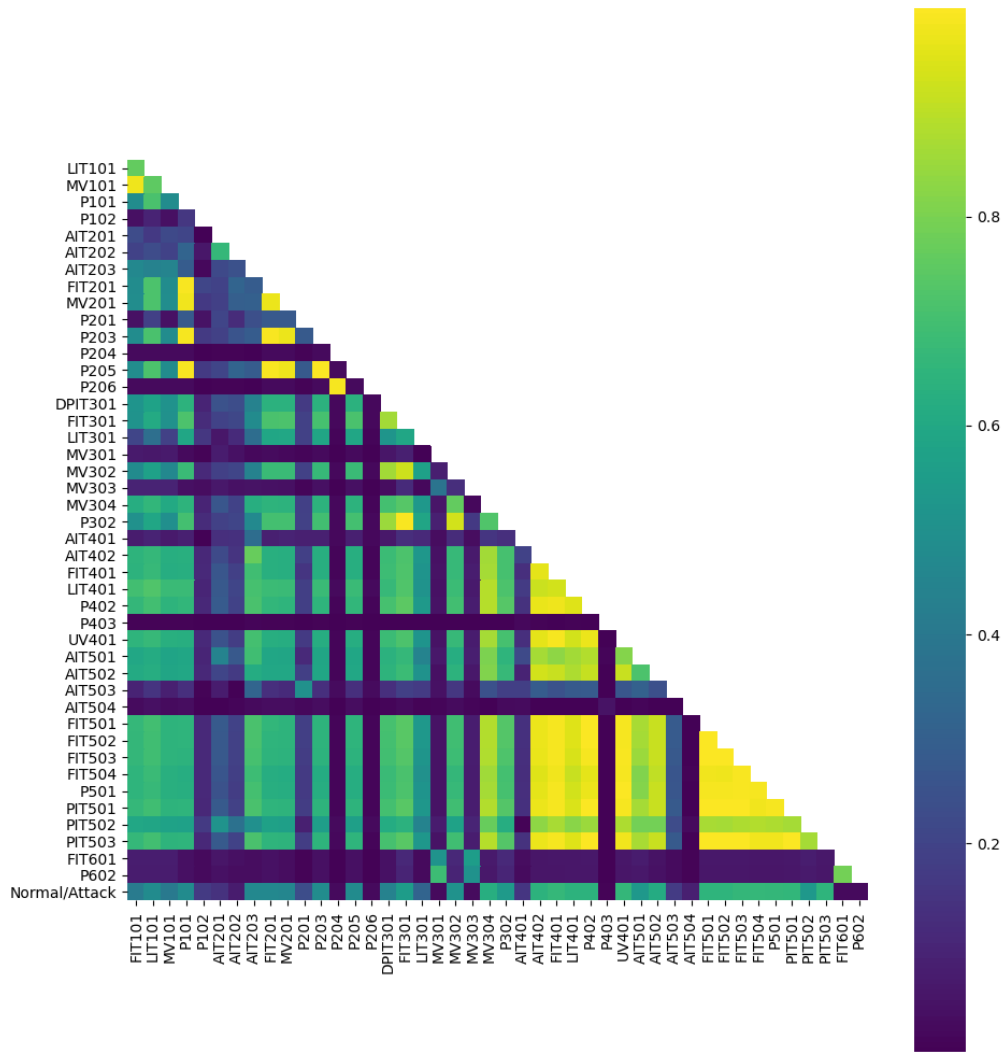


Figure 6.7.: Swat features and binary label as a correlation matrix

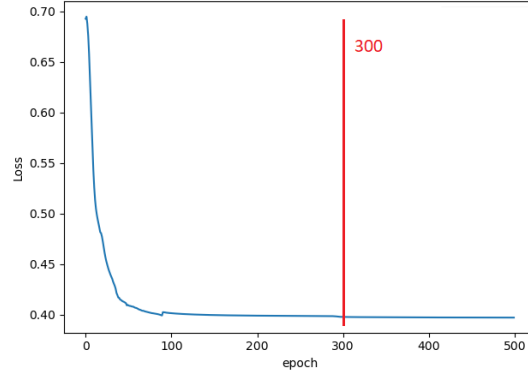


Figure 6.8.: Learning curve for the model which is going to be used for further swat adversarial experiments

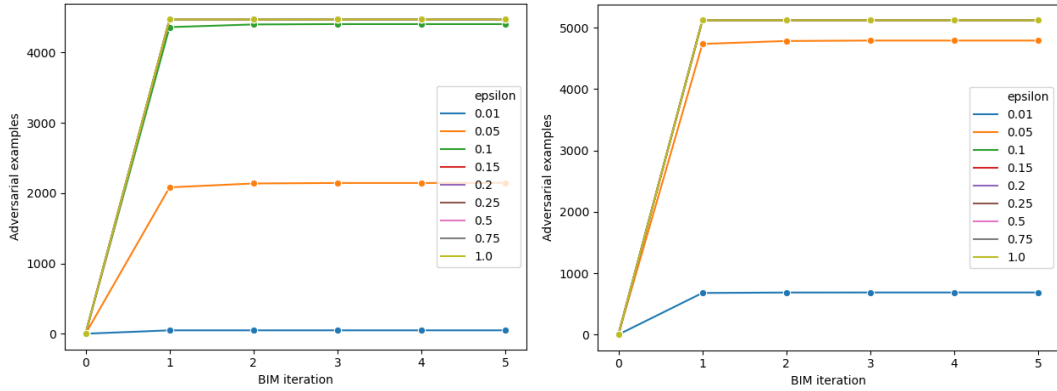


Figure 6.9.: Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the binary classification problem of the SWaT dataset

6.3.4. Adversarial attack and training

The epsilon value 0.05 were used for adversarial training. Figure 6.9 illustrates the results of the adversarial training and attacks. One can observe clearly the shift for the epsilon value 0.05 which indicates greater sensitivity to further adversarial attacks after adversarial training.

7. Summary

The final goal of this thesis consisted in exploring adversarial examples in a NIDS scenario. To achieve this much research was performed to implement functionalities and explore possibilities which were required to finalize the tasks one to three in table 1.1. This chapter summarizes the results derived from chapters 4 to 6.

7.1. Dataset preprocessing

In this sub-task an architecture was developed to reprocess the CICandMAL2017 dataset. This consists of preparing the *.pcap files by utilization with Argus and the generation of multiple datasets which can be used for training. The final amount of 12 datasets were generated with different properties and could be tested in binary and categorical classification scenario. It is also possible to perform ML learning tasks on the family label which was also generated in this process, but the amount of data points per label is too low for the generation of models which generalize well. Furthermore the SWaT dataset was preprocessed for further analysis. The preprocessing scripts can be used with any dataset which can be provided in the form *.pcap files, which are contained in a labelled directory structure. This makes the implementation to a tool which can be used to explore different configurations of such ML datasets.

7.2. NADICS 3 development

During this sub-task the NADICS framework was improved to ensure a modern and highly functional toolset for exploring security in network environments and ML. All changes were made in the classification part. Those included the usability improvements of label encoding and handling or training/testing table header interpretation. The preprocessing functions for handling imbalance in the dataset labels, PCA and feature selection were introduced. With the compare subroutine the performances of different ML models can be visualized and compared to each other.

The DNN framework PyTorch was added to the model portfolio by implementing an scikit-learn adapter. It can be utilized like other ML models with grid search. The implementation was kept flexible to ensure that different DNNs can be implemented in future. Furthermore the adversarial attacks FGSM and BIM were implemented.

7. Summary

	CICandMAL2017								SWaT			
	binary				category				binary			
Trained	benign	benign	adv.	adv.	benign	benign	adv.	adv.	benign	benign	adv.	adv.
Tested	benign	adv.	benign	adv.	benign	adv.	benign	adv.	benign	adv.	benign	adv.
Accuracy	0.97	0.95	0.97	0.95	0.85	0.57	0.79	0.78	0.92	0.77	0.95	0.95
f1 score	0.97	0.95	0.97	0.95	0.85	0.57	0.77	0.75	0.92	0.77	0.95	0.95
Recall	0.97	0.95	0.97	0.95	0.85	0.57	0.79	0.76	0.92	0.77	0.95	0.95
Precision	0.97	0.95	0.97	0.96	0.85	0.57	0.78	0.76	0.93	0.77	0.95	0.96

Table 7.1.: Final model performance

7.3. ML Experiments

The first part of the tasks handled in this chapter consisted in feature selection for the both datasets. This was performed with the functionality which was implemented into NADICS in the scope of the previous chapter. In future one could explore other possibilities for dimensionality reduction. One option is to utilize PCA. The next step consisted in determining the performance of the datasets regarding classical ML models. This was especially important for the CICandMAL2017 dataset since it provided the metric for the decision of the corresponding option from the set generated during preprocessing. After the successful training of the models they were attacked with FGSM and BIM attacks. During the analysis of the retraining one could observe increasing sensibility to further adversarial attacks. This behaviour can be explored in future works.

Table 7.1 summarizes the results derived in this chapter. In the case of the binary classification problem of the CICandMAL2017 dataset the adversarial training had no observable effect. However in the category classification of the same dataset multiple deviations can be noted. It appears that the adversarial training had the effect of hardening the model against adversarial examples at the cost of decreasing accuracy for the benign test dataset. Furthermore one can notice perturbations in the adversarial trained models performance metrics. This can be an indicator for unevenly distributed tuple labels. This assumption implies the possibility that there is an uneven distribution of tuples which can be turned into adversarial examples per label. The main reason the CICandMAL2017 dataset performs poorly is most probably the low quantity in tuples available.

In the SWaT dataset the same sensitivity to adversarial attacks was observed on the adversarial trained model. However the adversarial training had an positive effect on both benign and adversarial test dataset performance. The classifier was hardened successfully against the adversarial examples generated by attacking the DNN which was trained on the benign dataset.

A. Feature tables

Argument	description
seq	Sequence number
rank	Non zeroed sequence number
saddr	Source IP address
daddr	Destination IP address
sport	Source port
dport	Destination port
trans	aggregation record count
flgs	flow flags which appear during the connection
dur	Duration of the connection
mean	average duration of aggregated records
stddev	standard deviation of aggregated records
sum	total accumulated durations of aggregated records
min	minimum duration of aggregated records
max	maximum duration of aggregated records
proto	transaction protocol
stos	source TOS byte value
dtos	destination TOS byte value
sdsb	source diff serve byte value
ddsb	destination diff serve byte value
sttl	src -> dst TTL value
dttl	dst -> src TTL value
pkts	total transaction packet count
spkts	src -> dst packet count
pkts	total transaction packet count
spkts	src -> dst packet count
dpkts	dst -> src packet count

Table A.1.: Argus fields which were considered 1/2

Argument	description
bytes	total transaction bytes
sbytes	src -> dst transaction bytes
dbytes	dst -> src transaction bytes
appbytes	total application bytes
sappbytes	src -> dst application bytes
dappbytes	dst -> src application bytes
pcr	producer consumer ratio
loss	pkts retransmitted or dropped
sloss	source pkts retransmitted or dropped
dloss	destination pkts retransmitted or dropped
ploss	percent pkts retransmitted or dropped
psloss	percent source pkts retransmitted or dropped
pdloss	percent destination pkts retransmitted or dropped
retrans	pkts retransmitted
sretrans	source pkts retransmitted
dretrans	destination pkts retransmitted
pretrans	percent pkts retransmitted
psretrans	percent source pkts retransmitted
pdretrans	percent destination pkts retransmitted
sgap	source bytes missing in the data stream
dgap	destination bytes missing in the data stream
dir	direction of transaction
swin	source TCP window advertisement
dwin	destination TCP window advertisement
stcpb	source TCP base sequence number
dtcpb	destination TCP base sequence number
tcprrt	TCP connection setup round-trip time, the sum of 'synack' and 'ackdat'
synack	TCP connection setup time, the time between the SYN and the SYN_ACK packets
ackdat	TCP connection setup time, the time between the SYN_ACK and the ACK packets
tcpopt	The TCP connection options seen at initiation

Table A.2.: Argus fields which were considered 2/2

A. Feature tables

No	Name	Type	Description
1	FIT-101	Sensor	Flow meter; Measures inflow into raw water tank.
2	LIT-101	Sensor	Level Transmitter; Raw water tank level.
3	MV-101	Actuator	Motorized valve; Controls water flow to the raw water tank.
4	P-101	Actuator	Pump; Pumps water from raw water tank to second stage.
5	P-102 (backup)	Actuator	Pump; Pumps water from raw water tank to second stage.
6	AIT-201	Sensor	Conductivity analyser; Measures NaCl level.
7	AIT-202	Sensor	pH analyser; Measures HCl level.
8	AIT-203	Sensor	ORP analyser; Measures NaOCl level.
9	FIT-201	Sensor	Flow Transmitter; Control dosing pumps.
10	MV-201	Actuator	Motorized valve; Controls water flow to the UF feed water tank.
11	P-201	Actuator	Dosing pump; NaCl dosing pump.
12	P-202 (backup)	Actuator	Dosing pump; NaCl dosing pump.
13	P-203	Actuator	Dosing pump; HCl dosing pump.
14	P-204 (backup)	Actuator	Dosing pump; HCl dosing pump.
15	P-205	Actuator	Dosing pump; NaOCl dosing pump.
16	P-206 (backup)	Actuator	Dosing pump; NaOCl dosing pump.
17	DPIT-301	Sensor	Differential pressure indicating transmitter; Controls the backwash process.
18	FIT-301	Sensor	Flow meter; Measures the flow of water in the UF stage.
19	LIT-301	Sensor	Level Transmitter; UF feed water tank level.
20	MV-301	Actuator	Motorized Valve; Controls UF-Backwash process.
21	MV-302	Actuator	Motorized Valve; Controls water from UF process to De-Chlorination unit.
22	MV-303	Actuator	Motorized Valve; Controls UF-Backwash drain.
23	MV-304	Actuator	Motorized Valve; Controls UF drain.
24	P-301 (backup)	Actuator	UF feed Pump; Pumps water from UF feed water tank to RO feed water tank via UF filtration.
25	P-302	Actuator	UF feed Pump; Pumps water from UF feed water tank to RO feed water tank via UF filtration.
26	AIT-401	Sensor	RO hardness meter of water.
27	AIT-402	Sensor	ORP meter; Controls the NaHSO ₃ dosing(P203), NaOCl dosing (P205).
28	FIT-401	Sensor	Flow Transmitter ; Controls the UV dechlorinator.
29	LIT-401	Actuator	Level Transmitter; RO feed water tank level.
30	P-401 (backup)	Actuator	Pump; Pumps water from RO feed tank to UV dechlorinator.
31	P-402	Actuator	Pump; Pumps water from RO feed tank to UV dechlorinator.
32	P-403	Actuator	Sodium bi-sulphate pump.
33	P-404 (backup)	Actuator	Sodium bi-sulphate pump.
34	UV-401	Actuator	Dechlorinator; Removes chlorine from water.
35	AIT-501	Sensor	RO pH analyser; Measures HCl level.
36	AIT-502	Sensor	RO feed ORP analyser; Measures NaOCl level.
37	AIT-503	Sensor	RO feed conductivity analyser; Measures NaCl level.
38	AIT-504	Sensor	RO permeate conductivity analyser; Measures NaCl level.
39	FIT-501	Sensor	Flow meter; RO membrane inlet flow meter.
40	FIT-502	Sensor	Flow meter; RO Permeate flow meter.
41	FIT-503	Sensor	Flow meter; RO Reject flow meter.
42	FIT-504	Sensor	Flow meter; RO re-circulation flow meter.
43	P-501	Actuator	Pump; Pumps dechlorinated water to RO.
44	P-502 (backup)	Actuator	Pump; Pumps dechlorinated water to RO.
45	PIT-501	Sensor	Pressure meter; RO feed pressure.
46	PIT-502	Sensor	Pressure meter; RO permeate pressure.
47	PIT-503	Sensor	Pressure meter;RO reject pressure.
48	FIT-601	Sensor	Flow meter; UF Backwash flow meter.
49	P-601	Actuator	Pump; Pumps water from RO permeate tank to raw water tank (not used for data collection).
50	P-602	Actuator	Pump; Pumps water from UF back wash tank to UF filter to clean the membrane.
51	P-603	Actuator	Not implemented in SWaT yet.

Table A.3.: Feature Table of the SWAT data set [34]

B. CICandMAL2017 unexpected feature similarities

During training of classical ML algorithms on the CICandMAL2017 data set I faced feature to label similarities which did not fit in the scope of this thesis. Further analysis of those could be beneficial for the research of malware behaviour. Figure B.1 illustrates the similarity of labels to features. The feature `Dir_who` which corresponds to the occurrence of who request in the transmission history had a similarity of 0.2960 to the label "category_smsmalware" which is unexpected considering that in [32] it is stated that this category of malware does not communicate over Ethernet at all. There are three assumptions from my side which could explain this behaviour. The malware could actively precludes the device from sending arp requests or explore the network for vulnerabilities which can be detected with arp requests. Another explanation could be that there is an absence of the emulated user behaviour as stated in [32] in the infected phone which would result in less communication in general. The high similarity to the features `SrcTCPBase` and `DstTCPBase` which correspond to the base sequence numbers are unusual as well since those are chosen randomly in normal circumstances.

Figure B.2 shows the feature importances for the RandomForest classifier after features were dropped in the FS process for the dataset CICandMAL2017

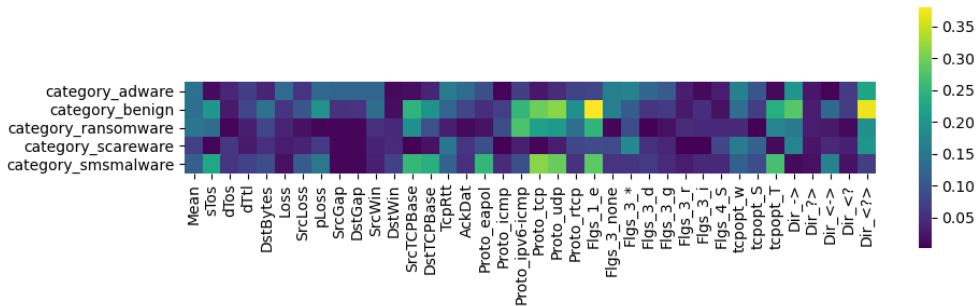


Figure B.1.: Similarity of distinct multi class labels to preselected features after feature selection based on similarity of the CICandMAL2017 data set.

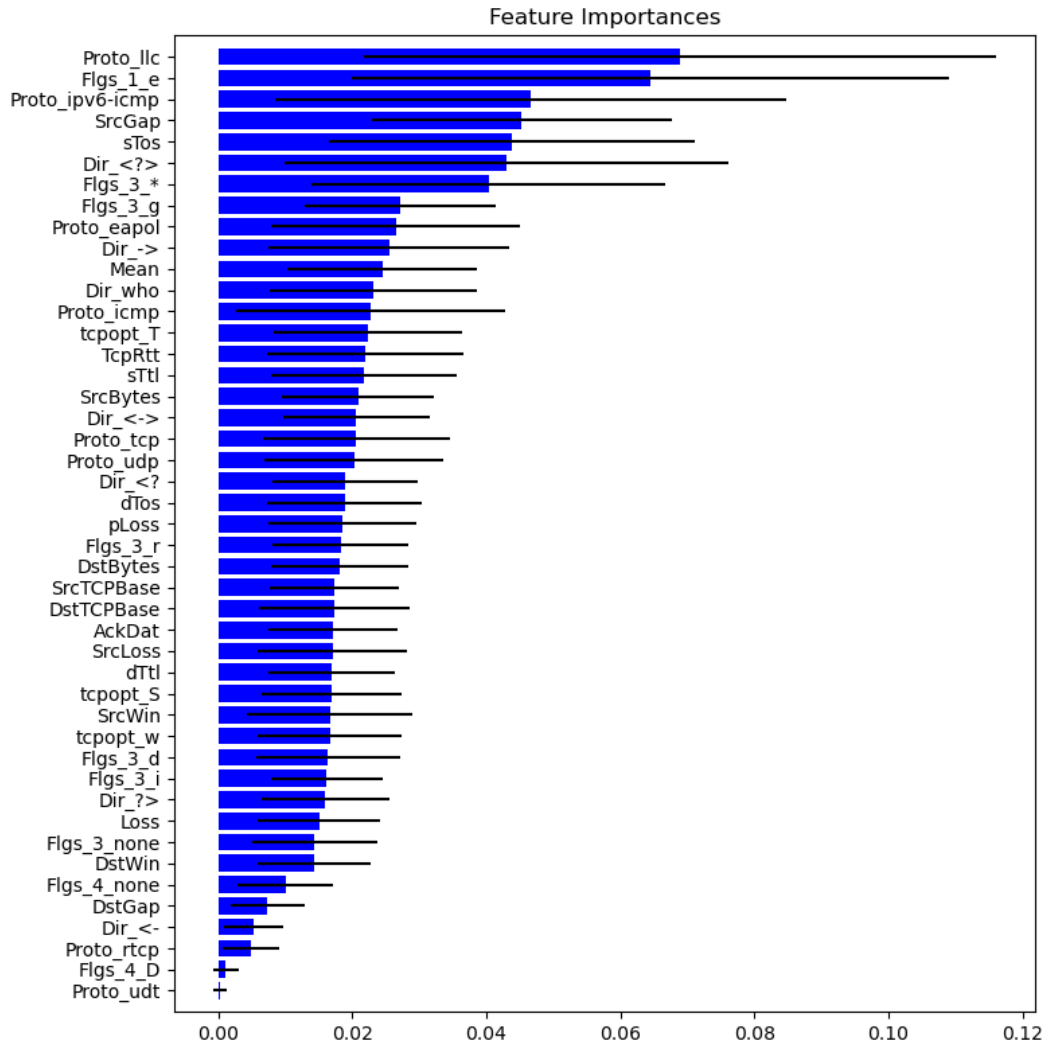


Figure B.2.: RF Feature importances in CICandMAL data set after dropping features based on similarity

C. Data splitting

After identifying the data sparsity problem mentioned in chapter 6.2.1 for the CICand-MAL2017 data set, I also explored the possibility to enrich the data set by splitting the pcap files in time windows. For this the *.pcap files were splitted before being processed by the *argus_csvs2train_csvs_v2.py* script resulting in more data tuples. However the new data set performed more poorly in combination with classical machine learning algorithms. This method could be explored further, especially after the good performances of the data set with classical ML algorithms, a trade off can be made between these two properties to generate more robust data sets.

List of Figures

2.1. Tcp connection sequence diagram	6
2.2. Left figure shows the distribution of samples for one feature in a binary classification scenario. The vertical line is the discriminator which classifies every sample on its left as negative and positive on the other side. The right image shows an example of an ROC curve curve. While the green line represents a random classifier and the blue the classifier resulting from the left figure. The red point represents an optimal classifier with 100% TP with 0% FP rate. By moving the discriminator function along the x axis on the left image the ratio between TN, FN, FP and FP changes which results in a movement on the blue ROC curve (right). The goal is to find an suitable trade-off between two axes.	12
2.3. (Left) A possible partition of the feature space with two features. the constants θ_1 and θ_2 are the learned weights of the classifier. (Right) the corresponding possible decision tree model.	17
2.4. To an omniscient decision tree (left) all features are available. In a random forest with feature randomness, each tree gets an unique feature subset. This ensures that the trained trees are also different, which contributes to the wisdom of the crowd effect.	18
2.5. An example for a k-NN hyperspace with 6 data points, two features and two classes. One can see that the hyperspace can be separated in two partition fairly easy with only few data points.	19
2.6. Commonly used activation functions	21
2.7. Adversarial attackers perspective after [1]. There are three main features which he has to extract after the enumeration phase. Those are subdivided in different attack approaches which are dependent on the information available to the attacker and his motivation.	25
2.8. Defense against adversarial attacks after [1]. The main classification of defenses against adversarial attack is determined by the question if the measures can be applied during training or after it.	27

3.1. The basic pipeline of NADICS. The input files are provided by the user and processed to xTrain, xTest, yTrain and yTest pandas data frames. The dataframes with x as prefix refer to the selected features which are used in the training and testing process. The dataframes with the prefix y provide the selected label correspondingly. After the training process the results can be stored in a textfile in the form of a line which provides information about accuracy and f1 score	31
4.1. The PCAP processing pipeline for the CICandMAL2017 dataset.	34
5.1. Different behaviour during training dependent on whether basic CV is used (Right) or not (Left)	41
5.2. Learning curves for the models which were trained with the k-fold CV. Binary classification (left) categorical (right)	42
6.1. Similarity matrix of the features from the CICandMAL2017 dataset app configuration with the binary label	46
6.2. Similarity of distinct labels to preselected features	47
6.3. Similarity of distinct labels to preselected features with dropped same connections	47
6.4. Epoch selection for DNN training for the CICandMAL2017 dataset labels binary (left) and category (right)	48
6.5. Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the binary classification problem of the CICandMAL2017 dataset	50
6.6. Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the category classification problem of the CICandMAL2017 dataset	51
6.7. Swat features and binary label as a correlation matrix	52
6.8. Learning curve for the model which is going to be used for further swat adversarial experiments	53
6.9. Final results for the adversarial attack of the original model attacked (left) and the retrained model (right) for the binary classification problem of the SWaT dataset	53
B.1. Similarity of distinct multi class labels to preselected features after feature selection based on similarity of the CICandMAL2017 data set.	59
B.2. RF Feature importances in CICandMAL data set after dropping features based on similarity	60

List of Tables

1.1. Thesis goals	3
2.1. OSI model	5
2.2. Classification of detection principles. Source: [10]	7
2.3. Caption	11
2.4. Scaling methods considered in this work with x as the input and x' as the scaled value	23
3.1. File sets which has to be provided to or are generated by NADICS	30
4.1. PCAP directory structure of the CICandMal2017 dataset	35
4.2. Variables which were removed from the CICancVAL2017 dataset before feature selection	37
6.1. FS based drop out of features for the CICandMAL2017 dataset	45
6.2. Classical ML learning results for the CICandMAL2017 dataset	49
6.3. FS based drop out of features for the SWaT dataset	51
6.4. Classical ML learning results for the SWaT dataset	51
7.1. Final model performance	55
A.1. Argus fields which were considered 1/2	56
A.2. Argus fields which were considered 2/2	57
A.3. Feature Table of the SWAT data set [34]	58

Acronyms

ANN	Artificial Neural Network. iii, iv, 1, 8, 20, 25, 32, 40
BIM	Basic Iterative Method. 42–44, 54, 55
CLI	Command-line Interface. 3, 29, 40
CPS	Cyber Physical System. 36
CSV	Comma Separated Values. 29
CV	Cross-validation. 41, 42, 63
DNN	Deep Neural Network. iii, 2, 3, 21, 23, 26, 40, 41, 44, 48, 54, 55, 63
DNS	Domain Name System. 35
FGSM	Fast Gradient Sign Method. 26, 42, 44, 54, 55
FN	False Negative. 12, 62
FP	False Positive. 12, 62
FPR	False Positive Rate. 11
FS	Feature Selection. 40, 44, 45, 51, 59, 64
GPS	Global Positioning System. 33
HMI	Human Machine Interface. 36
IDS	Intrusion Detection System. 1, 6–8
IP	Internet Protocol. 1, 4–6
IPS	Intrusion Prevention System. 6
MAC	Media Access Control. 4
MAP	Maximum A Posteriori. 14
MDP	Markov decision process. 9

ML Machine Learning. iv, 1–4, 8, 9, 13, 29, 32, 44, 48, 49, 51, 54, 55, 61, 64

MLE Maximum Likelihood Estimation. 15, 23

MLP Multi Layer Perceptron. 21

NADICS Network Anomaly Detection for Industrial Control Systems. iii, 2, 3, 29, 30, 33, 38–40, 54, 55

NIDS Network Intrusion Detection System. iii, iv, 1, 2, 6, 8, 44, 54

NTP Network Time Protocol. 35

OH encoding One-Hot encoding. 35, 36

OS Operating System. 4

OSI model Open Systems Interconnection model. 1, 4–6, 33

PCA Principal Component Analysis. 14, 39, 44, 54, 55

PLC Programmable Logic Controller. 8, 36

RFE Recursive Feature Elimination. 13

ROC curve Receiver Operating Characteristic curve. 11, 12, 62

SCADA Supervisory Control And Data Acquisition. 33, 36

SMS Short Message Service. 33

SNN Spiking Neural Network. 20

SVM Support Vector Machine. 20, 44

SWaT Secure Water Treatment. 33, 36

TCP Transmission Control Protocol. 5, 36

TN True Negative. 12, 62

TPR True Positive Rate. 11

UDP User Datagram Protocol. 5

VLAN Virtual LAN. 7

WAN World Area Network. 4–6, 33

Bibliography

- [1] M. Usama, J. Qadir, A. Al-Fuqaha, and M. Hamdi. *The Adversarial Machine Learning Conundrum: Can The Insecurity of ML Become The Achilles' Heel of Cognitive Networks?* URL: <http://arxiv.org/pdf/1906.00679v1>.
- [2] Tesla. *A tragic loss*. 2016 (accessed 12.11.2020). URL: https://www.tesla.com/de_DE/blog/tragic-loss?redirect=no.
- [3] C. Dartigue, H. I. Jang, and W. Zeng. "A New Data-Mining Based Approach for Network Intrusion Detection". In: *2009 Seventh Annual Communication Networks and Services Research Conference*. 2009, pp. 372–377.
- [4] E. Alhajjar, P. Maxwell, and N. D. Bastian. *Adversarial Machine Learning in Network Intrusion Detection Systems*. URL: <http://arxiv.org/pdf/2004.11898v1>.
- [5] Marek Pawlicki, Michał Choras, and Rafał Kozik. "Defending Network Intrusion Detection Systems against Adversarial Evasion Attacks". In: (2020).
- [6] Y. E. Sagduyu, Y. Shi, and T. Erpek. *IoT Network Security from the Perspective of Adversarial Deep Learning*. URL: <http://arxiv.org/pdf/1906.00076v1>.
- [7] Q. Liu, P. Li, W. Zhao, W. Cai, S. Yu, and V. C. M. Leung. "A Survey on Security Threats and Defensive Techniques of Machine Learning: A Data Driven View". In: *IEEE Access* 6 (2018), pp. 12103–12117. DOI: 10.1109/ACCESS.2018.2805680.
- [8] L. Goleniewski and K. W. Jarrett. *Telecommunications Essentials, Second Edition: The Complete Global Source (2nd Edition)*. Addison-Wesley Professional, 2006. ISBN: 0321427610.
- [9] L. Qian and B. E. Carpenter. "A flow-based performance analysis of TCP and TCP applications". In: *2012 18th IEEE International Conference on Networks (ICON)*. 2012, pp. 41–45. DOI: 10.1109/ICON.2012.6506531.
- [10] S. Axelsson. "Intrusion Detection Systems: A Survey and Taxonomy". In: (Apr. 2000).
- [11] U. Fiore, F. Palmieri, A. Castiglione, and A. de Santis. "Network anomaly detection with the restricted Boltzmann machine". In: *Neurocomputing* 122 (2013), pp. 13–23. ISSN: 09252312. DOI: 10.1016/j.neucom.2012.11.050.
- [12] K. P. Murphy. *Machine learning : a probabilistic perspective*. Cambridge, Mass. [u.a.]: MIT Press, 2013. ISBN: 9780262018029 0262018020. URL: https://www.amazon.com/Machine-Learning-Probabilistic-Perspective-Computation/dp/0262018020/ref=sr_1_2?ie=UTF8&qid=1336857747&sr=8-2.
- [13] C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006. ISBN: 0387310738.

- [14] I. Guyon and A. Elisseeff. "An Introduction of Variable and Feature Selection". In: *J. Machine Learning Research Special Issue on Variable and Feature Selection* 3 (Jan. 2003), pp. 1157–1182. doi: 10.1162/153244303322753616.
- [15] M. S. Kearns. "Thoughts on hypothesis boosting". In: 1988.
- [16] L. Breiman. "Prediction Games and Arcing Algorithms". In: *Neural Computation* 11.7 (1999), pp. 1493–1517.
- [17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [18] R. M. Williams and R. V. Yampolskiy. "Optical Illusions Images Dataset". In: (). URL: <https://arxiv.org/abs/1810.00415>.
- [19] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay. *Adversarial Attacks and Defences: A Survey*. URL: <https://arxiv.org/pdf/1810.00069.pdf> (visited on).
- [20] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. URL: <http://arxiv.org/pdf/1312.6199v4>.
- [21] B. Biggio and F. Roli. "Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning". In: *Pattern Recognition* 84 (2018), pp. 317–331. ISSN: 00313203. doi: 10.1016/j.patcog.2018.07.023. URL: <http://arxiv.org/pdf/1712.03141v2>.
- [22] I. J. Goodfellow, J. Shlens, and C. Szegedy. *Explaining and Harnessing Adversarial Examples*. URL: <http://arxiv.org/pdf/1412.6572v3>.
- [23] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial examples in the physical world*. URL: <http://arxiv.org/pdf/1607.02533v4>.
- [24] N. Carlini and D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. URL: <https://arxiv.org/pdf/1608.04644.pdf> (visited on).
- [25] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. URL: <http://arxiv.org/pdf/1706.06083v4>.
- [26] Matthias Hein and Maksym Andriushchenko. "Formal Guarantees on the Robustness of a Classifier against Adversarial Manipulation". In: ().
- [27] W. Xu, D. Evans, and Y. Qi. "Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks". In: *Proceedings 2018 Network and Distributed System Security Symposium*. Ed. by P. Traynor and A. Oprea. Reston, VA: Internet Society, February 18-21, 2018. ISBN: 1-891562-49-5. doi: 10.14722/ndss.2018.23198. URL: <https://arxiv.org/pdf/1704.01155.pdf> (visited on).
- [28] P. M. Nicolas Papernot, Xi Wu, Somesh Jha, and Ananthram Swami. "Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks". In: (2016). URL: <https://arxiv.org/pdf/1511.04508.pdf> (visited on).

- [29] C. Navarro, N. Hitschfeld, and L. Mateu. "A Survey on Parallel Computing and its Applications in Data-Parallel Problems Using GPU Architectures". In: *Communications in Computational Physics* 15 (Sept. 2013), pp. 285–329. doi: 10.4208/cicp.110113.010813a.
- [30] L. QoSient. *Argus manual*. 2012 (accessed 15.10.2020). URL: <https://qosient.com/argus/gettingstarted.shtml>.
- [31] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems* 32. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett. Curran Associates, Inc., 2019, pp. 8024–8035. URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [32] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani. "Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification". In: *2018 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 22/10/2018 - 25/10/2018, pp. 1–7. ISBN: 978-1-5386-7931-9. doi: 10.1109/CCST.2018.8585560.
- [33] iTrust. *Secure Water Treatment (SWaT) Testbed documentation*. 2018.
- [34] J. Goh, S. Adep, K. N. Junejo, and A. Mathur. "A Dataset to Support Research in the Design of Secure Water Treatment Systems". In: *CRITIS*. 2016.
- [35] S. U. of Technology and Design. *Secure Water Treatment (SWaT) Dataset*. 2015 (accessed 13.08.2020). URL: https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/.
- [36] PyTorch. *Adversarial Example Generation*. 2017 (accessed 17.11.2020). URL: https://pytorch.org/tutorials/beginner/fgsm_tutorial.html.